Department for Signal and Systems
University of Belgrade, School of Electrical Engineering

BACHELOR THESIS

STEREO VISION DEPTH ESTIMATION

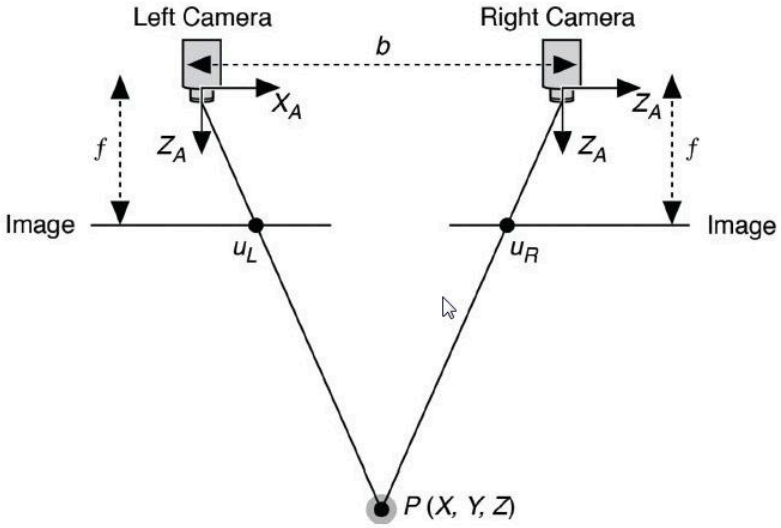
Author: Lukovic Petar 2019/0231
Mentor: asst. professor PhD Dragomir El Mezeni

Thesis presented for the degree of
Bachelor of Electrical Engineering and Computing

July 2023

Stereo Vision Depth Estimation

Author: Lukovic Petar 2019/0231
Mentor: asst. professor PhD Dragomir El Mezeni



A thesis presented for the degree of
Bachelor of Electrical Engineering and Computing

Department for Signal and Systems
University of Belgrade, School of Electrical Engineering

July 2023

Stereo Vision Depth Estimation

Abstract

This text covers one of the oldest, and most important, problems of computer vision, depth estimation. Starting from the modeling the camera, with linear model, through explaining camera parameters and lens deviation we will be able to completely understand every step of converting real world points into pixels. Then, we will take a glance at mathematical apparatus which helps us estimate depth from 2D image both using calibrated, or simple, stereo and uncalibrated stereo. At the end we will apply all mentioned processes on estimating depth in real time in high resolution video signal, using simple stereo for robotics purposes with help of Nvidia Jetson Nano™.

Contents

1 Introduction **3**

2 Camera model and Calibration **4**
2.1 Linear camera model 4
2.1.1 Sensor mapping 5
2.1.2 Intrinsic parameters 7
2.1.3 Extrinsic parameters 9
2.1.4 Projection matrix 10
2.2 Camera calibration 11
2.3 Distortion 15

3 Stereo vision depth estimation **19**
3.1 Estimating depth using simple stereo 20
3.1.1 Pixel matching in simple stereo 22
3.1.2 Simple stereo problems 25
3.2 Estimating depth using uncalibrated stereo 27
3.2.1 Stereo calibration and Fundamental matrix 27
3.2.2 Estimating fundamental matrix 32
3.2.3 Finding correspondences 33
3.2.4 Estimating depth 35
3.2.5 Stereo image rectification 36

4 Real time stereo vision depth estimation **38**
4.1 Required hardware 38
4.2 Algorithm implementation 40
4.2.1 Video and image acquisition 41
4.2.2 Camera calibration 42
4.2.3 Rectification and undistortion 46
4.2.4 Disparity calculation 48
4.3 Performance and Optimisation 53
4.4 Video depth estimation 55

5 Conclusion **57**

A Additional materials **58**
A.1 Calibration and test files: 58
A.2 Codes: 58

Chapter 1

Introduction

Much like our eyes, camera captures real world in three dimensions and present it in two dimensions. But, with one distinction. Our eyes, with help of brain, can perceive depth, whereas, this information of depth is lost in image capturing process. This text will try to help understand how we can achieve depth estimation through images.

Estimating depth is one of the oldest and most important problems of computer vision. So old in fact, that its exploration began as early as computer vision itself. So why are we so interested in estimating depth? Simple answer would be, depth information, besides color, is one of the most valuable characteristic of a scene. This is proved by numerous applications, ranging from constructing topographic maps, which is in fact first application of automated stereo vision depth estimation, 3D modeling and reconstruction to autonomous robot navigation or self driving, considering more modern applications.

Estimation techniques have developed much over the years, however they can still be separated into two big categories. First one is mono camera estimation, which focuses one camera to estimate depth using hidden image characteristic such as shading, contrast and colour. In modern day, mono camera methodology is executed using convolutional neural networks. Second category, which is topic of this text, utilises stereo camera system(calibrated and uncalibrated). Since one camera alone can not perceive depth, reasons will be explained later in the text, we take advantage of triangulation with two cameras to accurately calculate disparities, and later depth.

After covering theoretical foundation, we will take a look at one application of such algorithms. More precisely at real time video depth estimation using simple stereo camera pair and specialised hardware module Nvidia Jetson Nano™. This will close a loop on introducing stereo depth estimation techniques and mathematical apparatus to a reader, which is a sole purpose of this text.

Chapter 2

Camera model and Calibration

To be able to understand and accurately calculate depth using camera system we must understand how cameras operate. How we model them, and how we can map points in 3D world coordinates to pixels. Such as every phenomenon camera can be modeled in linear and non-linear fashion. Here, linear model is preferred, for its simplicity, mathematically, and more importantly numerically. Later into this chapter, we will also explain how non-linear phenomenons that inextricably occur can be incorporated into this linear model. Phenomenons such as lens distortion and its variations. Then, we will cover how these parameters can be extracted from simple templates, in process called calibration. Shortly this chapter is directed to readers that are not familiar with camera model and calibration, their peculiarities and problems, other reader may skip this chapter and proceed to Chapter 3, which explains depth estimation in detail.

2.1 Linear camera model

As mentioned, preferred way to model a camera is linear. Now what we mean when we say linear? We mean set of matrices, or one matrix that transforms 3D world coordinate system point P to its 2D projection on image plane in millimeters or pixels. However, before we derive these matrices, take a look at the figure below.

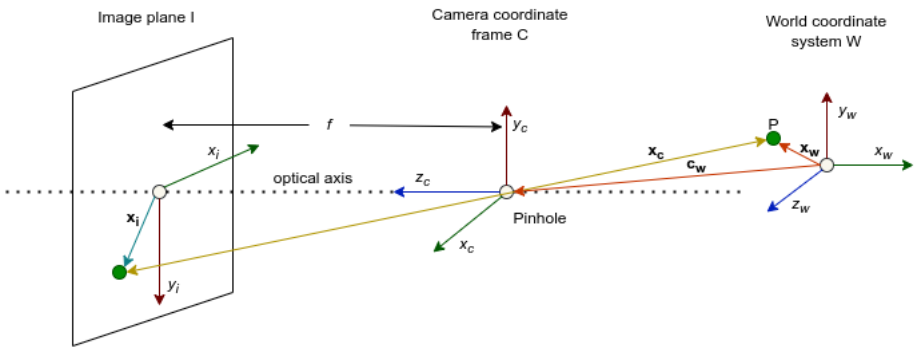
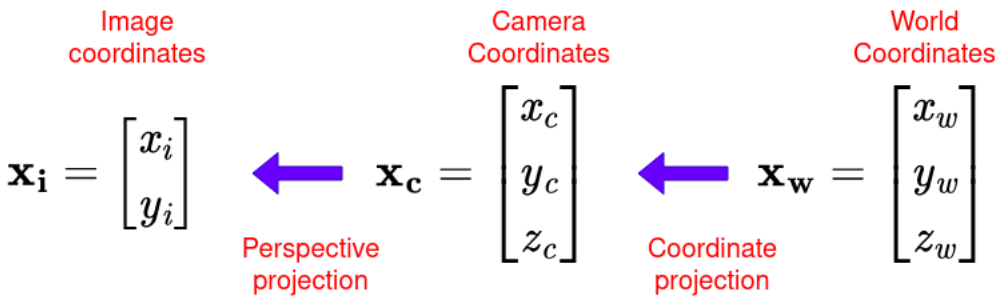


Figure 2.1: Camera model

Firstly, lets explain elements of camera model we need to derive transformation matrix. We divide world where image is created into three separate coordinate systems. World coordinate system, where point of projection is positioned. Camera coordinate system, system in which origin pinhole point is located, i.e. point where all rays intersect. And lastly, image plane, system where point P is projected. Unlike first two, image plane is 2d, whereas others are 3D coordinate systems. Also, on this figure image plane system is in millimeters. This is important because at the end we will derive transformation that transform world coordinates to image coordinates. Soon after we will discuss how we translate millimeters to pixels knowing some simple camera parameters. It is important that image plane is located exactly on length f along the optical axis. Optical axis is axis along which camera has rotational symmetry, i.e. passes through optical centre of the camera principal point of an image plane. Other variables that are shown are position vectors. From right to left they are: $\mathbf{x_w}$(position of projection P point in world coordinates), $\mathbf{x_c}$(position of projection point P in camera coordinates), $\mathbf{c_w}$(position of camera coordinate system origin in world coordinates) and lastly $\mathbf{x_i}$(position of projection point P in image coordinates).

Now that we have explained every variable that will be used, we can formalise the problem. Equations below shows what are the transformations needed to derive linear camera model. These are coordinated transformation matrix(transforms world coordinates to camera coordinates), and perspective projection matrix(transforms camera coordinates to image coordinates). Two matrices are also knows as extrinsic matrix and intrinsic matrix, respectively, and their elements extrinsic and intrinsic parameters.



2.1.1 Sensor mapping

Before we explain how we derive these transformations, lets take a look what parameter are characteristical to a camera and how we shift image coordinates in millimeters to image coordinates in pixels.

The most important parameters that are essential for a camera are its focal length *f*, height in pixel, height in millimeters, width in pixels and width in mil-

limeters. Last four parameters are often combined into two parameters. These are: $\mathbf{m_x}$(pixel density along x axis in pixels per millimeter) and $\mathbf{m_y}$(pixel density along y axis in pixels per millimeter). We use pixel densities along both axes since pixels are usually rectangular and not square. Other two parameters are principal point coordinates in pixels ($\mathbf{o_x}$, $\mathbf{o_y}$). These represent an approximation of a coordinates where optical axis pierces image sensor in pixels. We specify them since image sensor coordinate system(in pixels) in translated into top-left corner of a sensor due to easier processing(indexing).

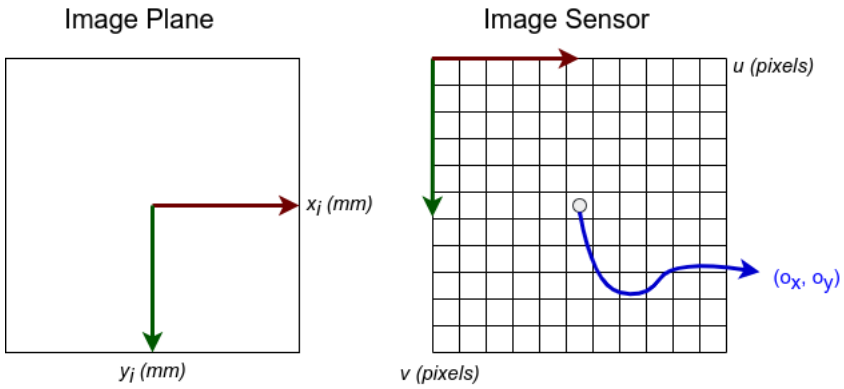


Figure 2.2: Sensor mapping

To derive further how camera coordinates are translated onto image plane in pixels we take a look on camera model figure above. Using triangle similarity law we have:

$$\frac{x_i}{f} = \frac{x_c}{z_c} \quad and \quad \frac{y_i}{f} = \frac{y_c}{z_c}$$

Which can be rewritten as following:

$$x_i = f \cdot \frac{x_c}{z_c} \quad and \quad y_i = f \cdot \frac{y_c}{z_c}$$

Now lets assume that image sensor does not have an offset coordinate system(origin is the centre of the sensor). We can say that coordinate u and v can be calculated as:

$$u = m_x \cdot f \cdot \frac{x_c}{z_c} \quad and \quad v = m_y \cdot f \cdot \frac{y_c}{z_c}$$

Here we have taken into consideration that pixel densities are not equal along two axes. Now, we can easily shift image sensor axes into top-left corner using principal point coordinates. This produces following equations:

$$u = m_x \cdot f \cdot \frac{x_c}{z_c} + o_x \quad and \quad v = m_y \cdot f \cdot \frac{y_c}{z_c} + o_y$$

Sometimes product of pixel density and focal length is labeled as $\mathbf{f_x}$ or $\mathbf{f_y}$, depending on the axes. These are called focal lengths in pixels along x and y axis

respectively. Incorporating this we have final equation for image sensor coordinates u and v in pixels:

$$u = f_x \cdot \frac{x_c}{z_c} + o_x \quad and \quad v = f_y \cdot \frac{y_c}{z_c} + o_y$$

Where, as mentioned:

$$(f_x, \ \ f_y) = (m_x \cdot f, \ \ m_y \cdot f)$$

2.1.2 Intrinsic parameters

This section is about trying to derive perspective projection transformation matrix, otherwise known as intrinsic matrix, and explaining its parameters. This matrix shows what is happening inside a camera when point is projected onto image sensor through a camera. Since we have already derived some equations that describe how to transform camera coordinates to image plane pixel coordinates, lets use them:

$$u = f_x \cdot \frac{x_c}{z_c} + o_x \quad and \quad v = f_y \cdot \frac{y_c}{z_c} + o_y \tag{2.1}$$

Looking into two equations we see that pixel coordinates are certainly not linear function of camera coordinates. Since we would like them to be, is there a way to somehow transform them so they become linear system? Answer is homogeneous coordinates.

Math review: Homogeneous coordinates

Homogeneous coordinates are a way of transforming $n-dimensional$ space into $(n+1)-dimensional$ space. For example, expressing 2D Cartesian coordinates with 3D Cartesian coordinates. Before we discuss into further detail, lets take a look at one example of transforming 2D into 3D in the figure below.

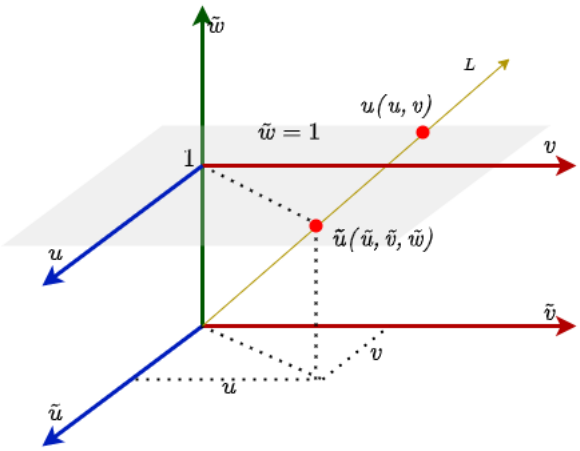


Figure 2.3: Homogeneous coordinates

Example shows how to transform point $u(u, v)$ in 2D, into point $\tilde{u}(\tilde{u}, \tilde{v}, \tilde{w})$ in 3D. where $\tilde{w} \neq 0$ is fictitious coordinate, which is usually set to be equal to one. Since homogeneous coordinates are derived from Cartesian coordinates, there is a relation between two coordinates systems. For 2D this is:

$$u = \frac{\tilde{u}}{\tilde{w}} \quad and \quad v = \frac{\tilde{v}}{\tilde{w}}$$

Theoretically speaking, coordinate $\tilde{w}$ can be equal to zero. This would mean that projected point is in infinity. Then other two homogeneous coordinates $\tilde{u}$ and $\tilde{v}$ are there just to point to the direction of the infinity. For example, take $\tilde{u} = 1$ and $\tilde{v} = 0$ and $\tilde{w} = 0$. This would mean that we try to project point with Cartesian coordinates of infinity along u axis. To make this more clear. one should try to lower plane $\tilde{w} = 1$ shown above to a plane $\tilde{w} = 0$, it would seem as the projection line L crosses plane in infinity.

Returning back to a way to represent 3D homogeneous coordinates with 2D Cartesian coordinates, and combining that with equations above (last matrix is derived for $\tilde{w} = 1$), we have:

$$u \equiv \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} \equiv \begin{bmatrix} \tilde{w} \cdot u \\ \tilde{w} \cdot v \\ \tilde{w} \end{bmatrix} \equiv \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

Now that mathematical formalism is covered, what is the physical explanation for homogeneous coordinates? From the equation above, it is obvious that for different $\tilde{w}$ we get the same Cartesian coordinates u and v. Meaning that for one 2D coordinates pair, we get infinitely many homogeneous points. It turns out that all of these point lay on the same line in 3D, this line is labeled L in the example and is called projection line. The reason being that every homogeneous coordinate on that line projects on the same point in 2D system(except the origin). For every $\tilde{w}$ we can find such $\tilde{u}$ and $\tilde{v}$ that lay on this line. It is important to notice that 2D coordinate system is always placed in $\tilde{w} = 1$ plane, in order to get accurate projection in 2D. This is easily seen in the equations. All of these conclusions should lead us to notice how every ray of light (projection line L) is projected onto only one point on image plane. Due to this, homogeneous coordinates are sometimes called projection coordinates. Also this projection is where information of depth is lost in image capturing process.

Hopefully with adequate choice of $\tilde{w}$ we can successfully use homogeneous coordinates and evade using non-linear model. Looking at the equation (2.1) derived before, it seems that by choosing $\tilde{w} = z_c = 1$ we would get rid of non-linearity. Lets do exactly that:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \equiv \begin{bmatrix} z_c \cdot u \\ z_c \cdot v \\ z_c \end{bmatrix} \equiv \begin{bmatrix} f_x \cdot x_c + z_c \cdot o_x \\ f_y \cdot y_c + z_c \cdot o_y \\ z_c \end{bmatrix}$$

Expressing camera coordinates with 4D homogeneous coordinates, where $\tilde{w} = 1$ we finally have:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \equiv \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

Within this equation we can extract some matrices, that are characteristical and have their names in literature. First one $\mathbf{K_{3\times3}}$ is called calibration matrix, whereas $\mathbf{M_{int}}$ is called intrinsic matrix. Also, four unknown parameters $(\mathbf{f_x}, \mathbf{f_y}, \mathbf{o_x}, \mathbf{o_y})$ are called intrinsic parameters and they are result of a calibration process.

$$\mathbf{K_{3x3}} = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \quad and \quad \mathbf{M_{int}} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

All these matrices and parameters describe how camera is projected on pixels. While the following section will describe how world on projected on camera we will rewrite final equations using homogeneous coordinates:

$$\tilde{\mathbf{u}} = [\mathbf{K_{3\times3}}|\mathbf{0}] \cdot \tilde{\mathbf{x_c}} = \mathbf{M_{int}} \cdot \tilde{\mathbf{x_c}}$$

2.1.3 Extrinsic parameters

Similarly to intrinsic parameter, extrinsic parameters model how world coordinates are projected on camera using homogeneous coordinates. Looking at an example, we see that this transformation is purely expressing point P in camera coordinates instead of world coordinates. Using simple vector operations we have:

$$\mathbf{x_c} = \mathbf{R} \cdot (\mathbf{x_w} - \mathbf{c_w}), \quad \mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

Here $\mathbf{x_c}$ is a position of point P in camera coordinate system, $\mathbf{x_w}$ is position of point P in world coordinate system. Other two parameter are position of world coordinate system in camera coordinate system $\mathbf{c_w}$ and rotation of world coordinate system relative to camera coordinate system $\mathbf{R}$. Pair $(\mathbf{R}, \ \mathbf{c_w})$ are also called extrinsic camera parameter, since they are enough to represent what is essentially transformation from real world to camera, i.e. what is happening in front of a camera during image capturing process. Usually last equation is rewritten as following:

$$\mathbf{x_c} = \mathbf{R} \cdot \mathbf{x_w} - \mathbf{R} \cdot \mathbf{c_w} = \mathbf{R} \cdot \mathbf{x_w} + \mathbf{t}, \quad \mathbf{t} = -\mathbf{R} \cdot \mathbf{c_w}$$

Here vector $\mathbf{t}$ is translation vector along three axes of world coordinate system relative to camera coordinate system. In expanded form last equation becomes:

$$\begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \cdot \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

This equation tells us that vector $\mathbf{x_w}$ is firstly rotated so that axes of world coordinate system align with axes of camera coordinate system, then vector is simply translated into origin of camera coordinate system using vector $\mathbf{t}$. Since we expressed

intrinsic parameter in homogeneous coordinates, we will translate last equation in homogeneous coordinates accordingly. Finally we have:

$$\begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_x \\ r_{31} & r_{32} & r_{33} & t_x \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix} \quad or \quad \tilde{\mathbf{x_c}} = \mathbf{M_{ext}} \cdot \tilde{\mathbf{x_w}}$$

Where:

$$\mathbf{M_{ext}} = \begin{bmatrix} \mathbf{R_{3\times3}} & \mathbf{t} \\ \mathbf{0_{1\times3}} & 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_x \\ r_{31} & r_{32} & r_{33} & t_x \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

With homogeneous coordinates we have achieved to express this transformation with only one matrix, this matrix is called extrinsic matrix and it is often labeled as $\mathbf{M_{ext}}$. Solely using this matrix we can translate a projected point from world to camera coordinates, and later to image coordinates using $\mathbf{M_{int}}$.

2.1.4 Projection matrix

Now that we know how to describe processes that happen both outside and inside of a camera during point projection onto a sensor. We can couple two matrices $\mathbf{M_{int}}$ and $\mathbf{M_{ext}}$ into one, called projection matrix $\mathbf{P_{3\times4}}$.

$$\tilde{\mathbf{u}} = [\mathbf{K_{3\times3}}|\mathbf{0}] \cdot \tilde{\mathbf{x_c}} = \mathbf{M_{int}} \cdot \tilde{\mathbf{x_c}} \quad and \quad \tilde{\mathbf{x_c}} = \begin{bmatrix} \mathbf{R_{3\times3}} & \mathbf{t} \\ \mathbf{0_{1\times3}} & 1 \end{bmatrix} \cdot \tilde{\mathbf{x_w}} = \mathbf{M_{ext}} \cdot \tilde{\mathbf{x_w}}$$

$$\mathbf{P_{3\times4}} = \mathbf{M_{int}} \cdot \mathbf{M_{int}} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix}$$

$$\tilde{\mathbf{u}} = \mathbf{P_{3x4}} \cdot \tilde{\mathbf{x_w}} \quad or \quad \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \cdot \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

This matrix is a result of a calibration process which will be discussed in greater detail in the following sections. What is important is that using matrix $\mathbf{P_{3\times4}}$ we can calculate projection in pixels for every point in world coordinate system. Also, using upper triangular property of calibration matrix, we can use QR factorisation to separate $\mathbf{M_{int}}$ and $\mathbf{M_{ext}}$ from $\mathbf{P_{3\times4}}$. This process is incorporated within most modern calibration algorithms, so the output of these kind of procedures are often all three camera matrices.

2.2 Camera calibration

If one is to explain camera calibration process in a few words, it would be reconstructing projection matrix **P** using object of known geometry. Lets firstly remind ourselves how world coordinates are projected onto image pixels using projection matrix:

$$\begin{bmatrix} u^{(i)} \\ v^{(i)} \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \cdot \begin{bmatrix} x_w^{(i)} \\ y_w^{(i)} \\ z_w^{(i)} \\ 1 \end{bmatrix}$$

Looking at this matrix equation, one could ask can we invert a process of projection and use known points in world coordinates and image plane to reconstruct back projection matrix? The answer is yes, we can. This is how calibration process is executed in industry applications. We would take some object of known geometry, take a picture of it, and use characteristical points to solve matrix equation. This object of known geometry is often a square, preferably with checkered pattern, such as shown in the figure below. If we position world coordinate system in the nearest square corner, we can easily calculate homogeneous world coordinates $\tilde{\mathbf{x}}_\mathbf{w}^{(\mathbf{i})}$ of points such as square corners or corner between black and white checks, if pattern is checkered. We choose these points since we can easily find which pixel(homogeneous) coordinates $\tilde{\mathbf{u}}^{(\mathbf{i})}$ correspond to each point in world coordinates. Finding these corresponding points in an image can be done both manually, by clicking on the image, or automatically using some kind of algorithm of edge detection.

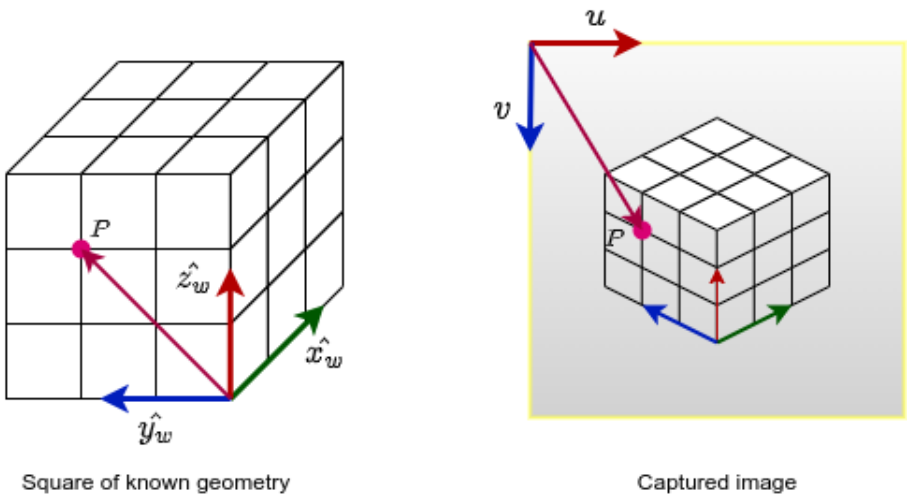


Figure 2.4: Calibration pattern

Take and example of point P pointed by purple vector in the figure above. If square tiles are all equal and have a dimension of 1cm, we can easily figure out that world coordinates of the point are $\mathbf{x}_\mathbf{w}^{(\mathbf{i})} = (0cm, 2cm, 2cm)$. Then, by clicking on the

image, we found out that $\mathbf{u^{(i)}} = (123px, 45px)$. We can take this point and use it to calculate the projection matrix. But one point is obviously not enough to calculate whole matrix. How many point is actually needed, we will find out after some math. Let us expand projection equation for point i:

$$u^{(i)} = \frac{p_{11}x_w^{(i)} + p_{12}y_w^{(i)} + p_{13}z_w^{(i)} + p_{14}}{p_{31}x_w^{(i)} + p_{32}y_w^{(i)} + p_{33}z_w^{(i)} + p_{34}}$$

$$v^{(i)} = \frac{p_{21}x_w^{(i)} + p_{22}y_w^{(i)} + p_{23}z_w^{(i)} + p_{24}}{p_{31}x_w^{(i)} + p_{32}y_w^{(i)} + p_{33}z_w^{(i)} + p_{34}}$$

If we rearrange these two expressions and combine them for n points, we get matrix equation such as(for each point in n points we have two equation for both coordinates $u^{(i)}$ and $v^{(i)}$):

$$\mathbf{A_{2n\times 12}} \cdot \mathbf{p_{12\times 1}} = \mathbf{0_{2n\times 1}}$$

Here matrices $\mathbf{A}$ and $\mathbf{p}$:

$$= \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} & p_{21} & p_{22} & p_{23} & p_{24} & p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix}^T$$

$$\mathbf{A} = \begin{bmatrix} x_w^{(1)} & y_w^{(1)} & z_w^{(1)} & 1 & 0 & 0 & 0 & 0 & -u_1x_w^{(1)} & -u_1y_w^{(1)} & -u_1z_w^{(1)} & -u_1 \\ 0 & 0 & 0 & 0 & x_w^{(1)} & y_w^{(1)} & z_w^{(1)} & 1 & -v_1x_w^{(1)} & -v_1y_w^{(1)} & -v_1z_w^{(1)} & -v_1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_w^{(i)} & y_w^{(i)} & z_w^{(i)} & 1 & 0 & 0 & 0 & 0 & -u_ix_w^{(i)} & -u_iy_w^{(i)} & -u_iz_w^{(i)} & -u_i \\ 0 & 0 & 0 & 0 & x_w^{(i)} & y_w^{(i)} & z_w^{(i)} & 1 & -v_ix_w^{(i)} & -v_iy_w^{(i)} & -v_iz_w^{(i)} & -v_i \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_w^{(n)} & y_w^{(n)} & z_w^{(n)} & 1 & 0 & 0 & 0 & 0 & -u_nx_w^{(n)} & -u_ny_w^{(n)} & -u_nz_w^{(n)} & -u_n \\ 0 & 0 & 0 & 0 & x_w^{(n)} & y_w^{(n)} & z_w^{(n)} & 1 & -v_nx_w^{(n)} & -v_ny_w^{(n)} & -v_nz_w^{(n)} & -v_n \end{bmatrix}$$

Looking at this matrix equation we see that we have twelve unknown parameters p_{ij}, meaning we need only six points on the square to calculate projection matrix or twelve independent equations.

Now that we have the equation to solve, how do we do it? We know that we expresses points in homogeneous coordinates, meaning that we can calculate projection matrix up to a scale factor. Reason for this is nature of homogeneous coordinates. Namely, when using world coordinates of a point on a square, we assumed that $\tilde{w} = 1$ which is somewhat incorrect. Generally, coordinate $\tilde{w}$ could be any number. If we remember homogeneous coordinates overview from beginning of this chapter, we know that one point in world coordinate transforms in a line in homogeneous coordinates, depending on the $\tilde{w}$. More accurate representation of projective transformation would be:

$$\begin{bmatrix} \tilde{w}_1u^{(i)} \\ \tilde{w}_1v^{(i)} \\ \tilde{w}_1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \cdot \begin{bmatrix} \tilde{w}_2x_w^{(i)} \\ \tilde{w}_2y_w^{(i)} \\ \tilde{w}_2z_w^{(i)} \\ \tilde{w}_2 \end{bmatrix}$$

Or in other terms:

$$\begin{bmatrix} u^{(i)} \\ v^{(i)} \\ 1 \end{bmatrix} = \frac{\tilde{w}_2}{\tilde{w}_1} \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \cdot \begin{bmatrix} x_w^{(i)} \\ y_w^{(i)} \\ z_w^{(i)} \\ 1 \end{bmatrix}$$

We have no way of knowing which point of the homogeneous line our coordinate project to, if we did know, we would not need to estimate depth in the first place. We will discuss this in greater detail when explaining how to estimate depth, but knowing to which pixel (u, v) our point in world projects, we can only calculate the ray on which this point lies, and not the exact point. For this reason we will have to use stereo vision to estimate depth. To explain this further, if we scale projection matrix, we are really scaling the world and the camera (projection matrix is product of camera and world properties in form of intrinsic and extrinsic matrices), we shall get exactly the same image. So what scale $K = \frac{\tilde{w}_2}{\tilde{w}_1}$ we should use? there are two options:

1. Take any parameter p_{ij} and fix its value. For example, $p_{34} = 1$
2. Optimise all parameters p_{ij} such that $\|p\| = 1$

With help of some math, second option is usually preferred one. Procedure is the following, find $\mathbf{p}$ that minimizes product $\mathbf{A_{2n\times 12}} \cdot \mathbf{p_{12\times 1}}$(try to make it equal to zero), such that $\|p\|^2 = 1$. In mathematical terms:

$$min_p(\|\mathbf{A} \cdot \mathbf{p}\|^2) \quad while \quad \|p\|^2 = 1$$

Considering matrices $\mathbf{p}$ and $\mathbf{A}$ are not square we have:

$$min_p(\mathbf{p}^T\mathbf{A}^T\mathbf{A}\mathbf{p}) \quad while \quad \mathbf{p}^T\mathbf{p} = 1$$

Now using Lagrange multipliers, we can try and find minimum of $\mathbf{p}^T\mathbf{A}^T\mathbf{A}\mathbf{p}$ with proposed constraint $\mathbf{p}^T\mathbf{p} = 1$. Solution to our problems is a are matrices $\mathbf{A}$ and $\mathbf{p}$ that minimize Lagrangian loss function. This function has form:

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda g(\mathbf{x})$$

Functions $f(\mathbf{x})$ and $g(\mathbf{x})$ are function to minimize and constraint function respectively. Which in our case is makes loss function to become:

$$L(\mathbf{p}, \lambda) = \mathbf{p}^T\mathbf{A}^T\mathbf{A}\mathbf{p} - \lambda(\mathbf{p}^T\mathbf{p} - 1)$$

Since loss function is a multi-variable function, to minimize it, derivatives along all variables must be equal to zero, including λ. Since, derivative along λ is the same equation as our constrain function, it is left to calculate derivative along vector $\mathbf{p}$ and equal it to zero. This gives us:

$$2\mathbf{A}^T\mathbf{A}\mathbf{p} - \lambda\mathbf{p} = 0$$

$$2\mathbf{A}^T\mathbf{A}\mathbf{p} = \lambda\mathbf{p}$$

Looking at this expression, it should remind us of eigenvalue problem. Which it exactly is for square matrix $\mathbf{A}^T\mathbf{A}$. So the solution to finding projection matrix under this constraint is eigenvector $\mathbf{p}$ such that its eigenvalue λ is smallest. According to linear algebra, smaller the eigenvalue, closer the product of eigenvector and its matrix is to zero. This is according to expression which we use to calculate eigenvector. In general form this process is given by $(\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = 0$. This is exactly what we wanted. When we have found adequate $\mathbf{p}$, it is only left to rearrange it into projection matrix $\mathbf{P}$.

$$\mathbf{p_{12\times 1}} = \begin{bmatrix} p_{11} \\ p_{12} \\ p_{13} \\ \vdots \\ p_{34} \end{bmatrix} \implies \mathbf{P_{3\times 4}} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix}$$

What is left how to extract other relevant camera information from projection matrix. These include $\mathbf{K}$, $\mathbf{R}$ and $\mathbf{t}$, or calibration matrix, rotation matrix and translation matrix. We know:

$$\begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_x \\ r_{31} & r_{32} & r_{33} & t_x \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Extracting $\mathbf{K}$ and $\mathbf{R}$ from this equation we have:

$$\begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} = \mathbf{KR}$$

Considering that $\mathbf{K}$ is upper-triangular and $\mathbf{R}$ is orthogonal. As a reminder, row vectors of matrix $\mathbf{R}$ are orthogonal, i.e. their length is equal exactly 1 and row vectors are orthogonal. We know this since rotation matrix is a transformation of aligning one coordinate system(world) to another(camera) whose axes are orthogonal themselves, so rotation matrix must be orthogonal too. Now using $\mathbf{QR}$ factorization, which decouples any matrix to product of two matrices one of which is upper triangular and other one is orthogonal. It is only left to extract translation vector $\mathbf{t}$. We do this as such:

$$\begin{bmatrix} p_{14} \\ p_{24} \\ p_{34} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} t_x \\ t_x \\ t_x \end{bmatrix} = \mathbf{Kt} \implies \mathbf{t} = \mathbf{K}^{-1} \begin{bmatrix} p_{14} \\ p_{24} \\ p_{34} \end{bmatrix}$$

2.3 Distortion

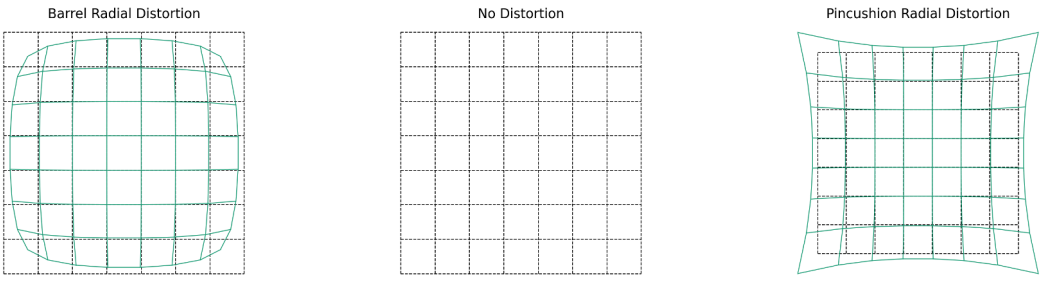
Deriving linear camera model we have assumed that camera has a pinhole and that all light goes through one point without distorting. In real applications this is not the case. Cameras have lenses or even systems of lenses combined together, and lenses are by no means a perfect pinhole. Light bends going trough them due to their shape or refraction, sometimes even their imperfection or their misalignment. Generally we can categorise distortion types into three categories. They are:

1. Symmetrical radial distortion
2. Asymmetrical radial distortion
3. Tangential(decentering) distortion

Indeed, a camera could have a combination of above-mentioned distortion types. This is sometimes referred as compound distortion. We do not cover this type of distortion since it can we broke apart into three listed categories and dealt with individually. Biggest problem of the distortion is the fact that lines that are straight in the scene, are no longer straight in the image. This causes problems in many applications, such as depth estimation which is the topic of this text. The idea of undistorting images is regaining this property. Also, worth mentioning, is that distorted images field of view is no longer rectangular.

Symmetrical radial distortion

This is the distortion type that we think of when we speaking of lens distortion, it is also the most common one. We differ two types, barrel or negative distortion and pincushion or positive distortion. Looking at the figure below, names are self-explanatory, however, we can discuss terms positive and negative. Positive distortion means that distance from centre of the lens increases, and vice versa. In the figure below we can see how these two types are manifested in relation to normal, non distorted image.



(a) Barrel distortion (b) No distortion (c) Pincushion distortion

Figure 2.5: Symmetrical radial distortion types

Origin of this type lays in the lens imperfection and shape. As light enters the lens outside of the perspective centre it bends towards the image plane. It might be easiest to think of symmetric radial distortion as if we were mapping the image plane to the convexity or concavity of the lens itself. Convex lens is to blame for barrel distortion since it converges light, whereas, concave lenses are a reason for pincushion distortion since they diverge light rays. In the end, name symmetrical radial distortion is given due to the fact that distortion is modeled just as a function of distance from centre of the image plane, along the radial direction. This causes distorted image to have radial symmetry.

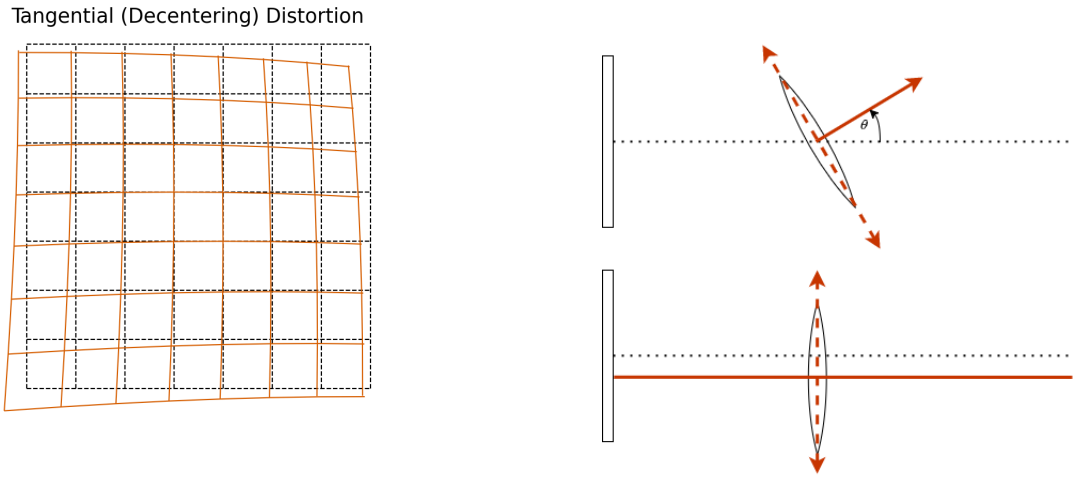
Asymmetrical radial distortion

Characteristic of asymmetrical radial distortion is that it is a function of both distance from image centre and distance of the object imaged. This is why images distorted with this type no longer posses radial symmetry. Due to the fact that asymmetrical distortion is a function of distance of an object, it is really hard to depict it, since it depends on the scene portrayed. However, we can enumerate some cases of its appearance:

1. Cameras with long focal lengths and very-short relative object distances. For example a very-near-field telephoto lens that is capturing many objects very close

2. Observing objects through a medium of high-refraction, or differing refractive indices. For example, two objects underwater where one is near and one is far away

Tangential(decentering) distortion



(a) Tangential distortion example

(b) Origin of tangential distortion

Figure 2.6: Tangential(decentering) distortion

Contrary to already-mentioned distortion types, this one is caused by misalignment of lens in camera making process. Misalignment are of two types. Firstly, lens can be tilted around projection centre, secondly, it can be translated parallelly to the what should have been optical axis, see the figure at the beginning of this section. Due to the second problem, this distortion got its name, since it is decentralised from the optical axis. Optical system can have both of these misalignment issues, however, this distortion is usually quite small. We can imagine this distortion as tilting of an image or its translation similarly to what is shown in the figure above.

Modeling distortion

Distortion has many ways of being modeled. However, we try to model it so it is numerically acceptable. There is no point in over-complicated model if it can not be computed. In industry there are two models of lens distortion that are commonly used, or their variations for that matter. These are Brown-Conrady and Kannala-Brandt model, and both of them use polynomials to represent distortion. In this text we will take a look at Brown-Conrady distortion model, since it is more commonly used. For example, **openCV** distortion modeling uses this model, which is most used computer vision library. More about other types of distortion modeling reader can find looking at the bibliography materials.

We will model symmetrical radial distortion and tangential distortion separately. Lets start from the first mentioned. In order to simplify math, in both cases we will consider centre of the image plane to be centre of the (u, v) coordinate system. Result can easily be translated into top left image corner. As mentioned distortion will be modeled using $n + 2$-order polynomial as function of radial distance of the pixel. This gives us general form of distortion along radial axis:

$$\delta_r = k_1 r^3 + k_2 r^5 + k_3 r^7 + ... + k_n r^{n+2} \quad where \quad r = \sqrt{u^2 + v^2}$$

Considering that what is important is distortion along both (u, v) axes, lets use some basic trigonometry to project this vector:

$$\delta_{ur} = \sin(\psi)\delta_r = \frac{u}{r}(k_1 r^3 + k_2 r^5 + k_3 r^7 + ... + k_n r^{n+2})$$

$$\delta_{vr} = \cos(\psi)\delta_r = \frac{v}{r}(k_1 r^3 + k_2 r^5 + k_3 r^7 + ... + k_n r^{n+2})$$

Here angle ψ is angle of radial vector in radial coordinate system. Distortions are usually really small values, especially in modern times where lens production methods have improved vastly. For this reason, distortion correction algorithms usually use just first three or even two coefficients for modeling symmetrical radial distortions, even for more complex lens systems. Then equations become:

$$\delta_{ur} = \sin(\psi)\delta_r = u(k_1 r^2 + k_2 r^4 + k_3 r^6)$$

$$\delta_{vr} = \cos(\psi)\delta_r = v(k_1 r^2 + k_2 r^4 + k_3 r^6)$$

Before we plug this increments into our linear model, lets take a look at modeling tangential distortion. Then we will combine them together into model. According to Brown-Conrady model, tangential distortion is derived from more complex equation:

$$\delta_r = P(r)\cos(\psi - \psi_0)$$

Here ψ_0 is an angle in which direction tangential distortion is the largest. Similarly to radial distortion $P(r)$ is some sort of even-power polynomial series, and is often called decentering distortion profile function. Unwrapping this equation and projecting vector δ_r along its components, we get general form for both tangential distortion components:

$$\delta_{ut} = (p_1(r^2 + 2u^2) + 2p_2uv)(1 + p_2r^2 + p_4r^4 + p_5r^6 + ...)$$

$$\delta_{vt} = (p_2(r^2 + 2v^2) + 2p_1uv)(1 + p_2r^2 + p_4r^4 + p_5r^6 + ...)$$

Since tangential distortion is usually really small, even smaller that radial, we usually use just first two coefficients. Reason for this is that lenses are rarely that much decentralized or tilted so we have to use more complex models. We have final equations:

$$\delta_{ut} = p_1(r^2 + 2u^2) + 2p_2uv$$

$$\delta_{vt} = p_2(r^2 + 2v^2) + 2p_1uv$$

Now that we have expressed both important distortions, we can have them combined in our model:

$$u_{distorted} = u + \delta_{ut} + \delta_{ur} = u + [p_1(r^2 + 2u^2) + 2p_2uv] + [u(k_1r^2 + k_2r^4 + k_3r^6)]$$

$$v_{distorted} = u + \delta_{vt} + \delta_{vr} = v + [p_2(r^2 + 2v^2) + 2p_1uv] + [v(k_1r^2 + k_2r^4 + k_3r^6)]$$

Undistorting is done using following expressions and solving them for (u, v), whereas $u_{distorted}$ and $v_{distorted}$ are pixel position on the image captured. This process returns a image with pixel rearranged so the property of image where straight lines are straight is regained. In general, these two distortion are not big, especially if field of view of a camera system is small. This is the reason we use just five parameters $(k_1, k_2, k_3, p_1, p_2)$. They are often called distortion coefficients, and they can also be a result of camera calibration process along with intrinsic and extrinsic matrices, for example **openCV** library calculates these coefficients during calibration process. Also worth noting is the fact that modeling distortion removes linearity from the model, since we use non-linear functions of coordinates such as r.

Chapter 3

Stereo vision depth estimation

In the preceding chapter we have discussed ways to model a camera. We have derived equations that help us project points in world coordinates to pixels, or sort of. For this purpose two coordinates system were used, world and camera coordinate system. Depth estimation problem is focused on finding z component of a point in camera coordinates. This is completely intuitive since we want to estimate depth in relation to camera, i.e. pinhole, and not in relation to world coordinates. Additionally, we know that world coordinates are not fixed, or in any way related to a camera. In other words, we can place them anywhere in real world. During calibration process this is usually nearest calibration square corner. So why do we use world coordinates in the first place, if we do not use them for this problem. The idea of adding world coordinates is to be able to describe projected point in some coordinate system. Without knowing exactly where we the origin of world coordinates is we would not be able to project point in real world to image plane. Consider that we have no way of knowing where the point is relation to camera. This is why we artificially place world coordinate system where it is easy for us to locate characteristical points. In other words, regarding this problem whole projection matrix is not of interest, we use it just to be able to calibrate the camera and find intrinsic matrix, or more precisely calibration matrix $\mathbf{K}$.

But how do we use this calibration matrix to extract information of depth. Let us firstly look again at the imaging system in the figure below.

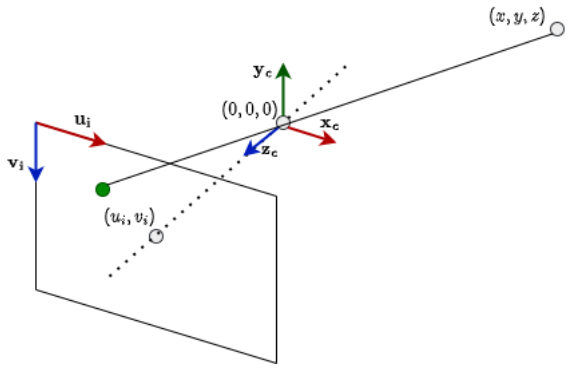


Figure 3.1: One camera depth mapping

Here we have shown image plane, camera coordinate system and point of interest in camera coordinates. What information can we derive from this setup? Using triangle similarity and the fact that image plane is located exactly at distance f along $\mathbf{z_c}$ axis, we have:

$$u = f_x \cdot \frac{x_c}{z_c} + o_x \quad and \quad v = f_y \cdot \frac{y_c}{z_c} + o_y$$

As a reminder, (f_x, f_y) are focal lengths along axes u and v in the image plane, and (o_x, o_y) are principal point coordinates along same axes(pixels where optical axis passes through image plane). These four parameters are part of intrinsic and calibration matrix, and can be extracted from them using:

$$\mathbf{K_{3x3}} = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \quad and \quad \mathbf{M_{int}} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Considering that calibration process is done according to steps explained in previous chapter, these two matrices are known to us. Pixels (u, v) where point is detected are also know(we want to calculate depth for this exact point). The problem here is that we have just two equations and three unknown variables(coordinates in relation to camera). So how do we surpass this? Using homogeneous coordinates we were able to evade this problem in order to calculate projection matrix, however we have not solved it. Projection matrix is known only up to a scale, and we cannot use it to calculate exact mapping from 3D to 2D in image plane. In other words, depth information is not lost in the process, it was always there, however we only lacked one equation to find it. What we can do is calculate an equation of a ray which is projected into wanted pixels (u, v), example of this ray is shown in the figure above. Knowing that intersection of two rays produces gives us a point, we can use two rays to find exactly from which point light is reflected. But where do we find second ray? Answer is from another camera(image), using triangulation. With two cameras, or stereo vision, we can extract exact point in space and find depth. How exactly do we do this, and where do we put second camera will be discussed in greater detail in the following two sections.

3.1 Estimating depth using simple stereo

Simple, calibrated or horizontal stereo is one way of getting about the problem described above. This is also a more simple solution to it, however more constrained. Namely, with this camera setup we just translate one camera along x(horizontal stereo) axis of the other camera, so the image sensors remain in the same plane. The distance between two origins is b which is called the baseline. Considering that we fixed the left camera, and translated the right one, it holds that the exact location of origin of the right camera is $(b, 0, 0)$ relative to the left camera coordinate system. So we utilize construction of the stereo system to guarantee us the location of one camera in relation to the other one, for this reason we call this simple or calibrated stereo. This does not hold true for the uncalibrated stereo as it will be discussed in the following section. In case of uncalibrated stereo, we will be responsible to find the exact orientation of right camera in relation to left camera. It is also important to note that cameras used in calibrated stereo system have to be identical, which is

along with its construction, the constrain of simple stereo mentioned above. Before deriving any equations, lets firstly take a look ot a figure below which shows typical configuration of simple stereo.

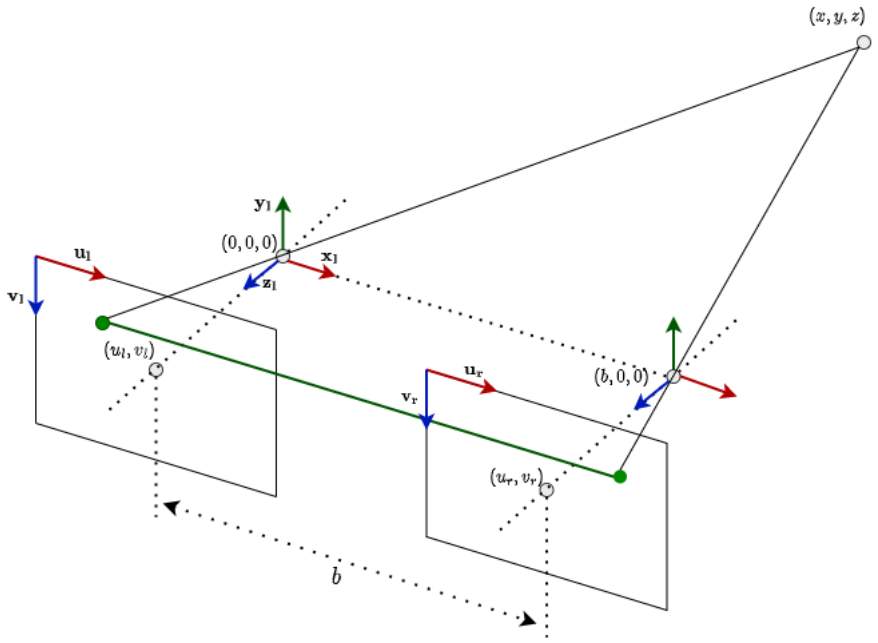


Figure 3.2: Simple stereo triangulation

As it was discussed, we can use two outgoing rays two calculated origin of the point projected onto both image planes. Lets write them down. For the left camera it holds:

$$u_l = f_x \cdot \frac{x_l}{z_l} + o_x \quad and \quad v_l = f_y \cdot \frac{y_l}{z_l} + o_y$$

and for the right camera:

$$u_r = f_x \cdot \frac{x_r}{z_r} + o_x \quad and \quad v_r = f_y \cdot \frac{y_r}{z_r} + o_y$$

Note that intrinsic parameter in all four equations are the same as we mentioned that the cameras are identical. Due to the construction of the simple stereo system described above we can easily conclude that it is true for every point in the right camera coordinate system that:

$$(x_l,\ y_l,\ z_l) = (x_r + b,\ y_r,\ z_r)$$

Given this property we can rewrite four equations as follows:

$$u_l = f_x \cdot \frac{x_l}{z_l} + o_x \quad and \quad v_l = f_y \cdot \frac{y_l}{z_l} + o_y$$

for the left camera:

$$u_r = f_x \cdot \frac{x_l - b}{z_l} + o_x \quad and \quad v_r = f_y \cdot \frac{y_l}{z_l} + o_y$$

and for the right camera. Now we have four equations and just the unknowns coordinates. Solving these equations for $(x_l,\ y_l,\ z_l)$ we are going to be able to find exact point where two rays intersect. Third coordinate z_l is the depth we are looking for, whereas other two coordinates are used in problems such as 3D reconstruction. Equations that give us three coordinates are:

$$x_l = \frac{b(u_l - o_x)}{u_l - u_r}$$

$$y_l = \frac{bf_x(v_l - o_x)}{f_y(u_l - u_r)}$$

$$z_l = \frac{bf_x}{u_l - u_r}$$

Note that due to the construction v_l and v_r are the same. Meaning that pixel corresponding to (u_l, v_l) in the left image can be located on the line $v_r = v_l$ on the right image. So we have used two rays to express equation give us exact location of the point in 3D of the left camera using right camera. We can find depth for every pixel on the left camera, but firstly we have to find its corresponding pixel on the right camera. Techniques for solving this issue will be discussed in detail in the following text. For now it is important that we can accurately calculate depth using two corresponding pixels on which arbitrary point in space is projected.

In expression that we use to calculate depth we can extract one characteristical difference, the difference of u coordinates of the corresponding pixels in two images. This difference is called disparity, and depth is inversely proportional to it. Greater the depth, smaller the difference, meaning that point in infinity should project into exactly the same pixel on both images. Rephrased, when we want to capture object very far away, we will get two exactly the same images. Additionally, disparity is proportional to baseline. This is quite intuitive given the fact that we expect the disparities to be bigger as we translate cameras farther from each other(disparities take value from greater range). In other words, if we wish to make depth calculation more accurate, we should enlarge baseline.

3.1.1 Pixel matching in simple stereo

Problem of estimating depth using simple stereo ends up in problem of finding corresponding pixels. This is usually done via template matching. Namely, purely by comparing pixel values would lead algorithm into many miss-matches. This is due to a fact that single pixel carries information that is too small. This is why we usually compare blocks surrounding compared pixels. Block are usually square, with constant size N, however their size can be adaptive. In generally, all algorithms can we summed up as taking one pixels in the left image and comparing it to the pixels in the right image. Which pixels on the right image are we taking into consideration depends on the algorithm used. Using this criterion we separate most common algorithms into three categories:

1. Block matching algorithms

2. Local matching algorithms

3. Neural networks matching algorithms

In the following text we will consider first category in detail, and briefly overview other two.

Block matching algorithms

All block matching algorithms, as name suggests, compare block around pixel on the left image with the block around pixel in the right image. These algorithms utilize the fact that in simple stereo it is guaranteed, because of the construction, that the corresponding pixel in the right image is always on the same v coordinate as the pixel in left image. So instead of comparing left pixel with all pixels on the right image, we compare only the pixels along the same, matching line. This reduces complexity of such algorithms from $height \times width$ to only $width$ comparisons for every pixel of the left image. See the figure below.

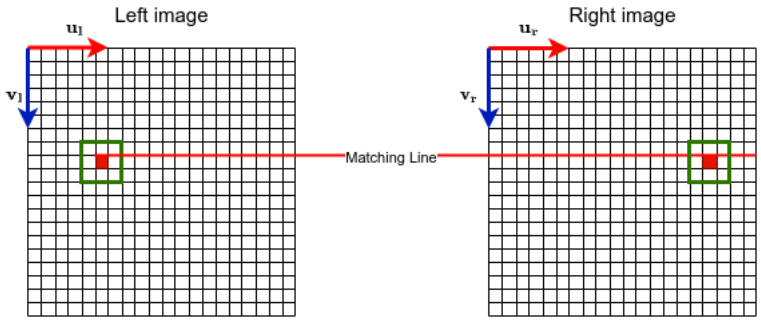


Figure 3.3: Pixel matching

Now, what is the metric of similarity of block matching algorithms. In use are three usual similarity metrics. First one is **sum of absolute differences**, which is given by the formula:

$$SAD(k,l) = \sum_{(i,j)\in T} |E_l(i,j) - E_r(i+k, j+l)|$$

Here k and l are pixel coordinates along the line of matching in the right image, T is template or block and E is pixel intensity. Idea is to subtract block on the left image with the each block on the right image.This is done by summing absolute differences pixel by pixel in the whole block. Smaller the difference, better the match. Second metrics is quite similar, and is called **sum of squared differences**. This metric is given by the equation:

$$SSD(k,l) = \sum_{(i,j)\in T} (E_l(i,j) - E_r(i+k, j+l))^2$$

This metric is also minimized, however, instead of using absolute of differences, we use square of differences. This is then summed up inside whole block. Generally, this metric is more used between the two even though it gives more importance to outliers. And the last metric is **normalized cross-correlation**. This way of comparing blocks is used, for example, when one camera has slightly different lighting. In this case it is important to normalize a metric since lighting affect pixel intensity and block may have too big of a difference between two images. We calculate normalized cross-correlation using following expression:

$$NCC(k,l) = \frac{\sum_{(i,j)\in T} |E_l(i,j) - E_r(i+k, j+l)|}{\sqrt{\sum_{(i,j)\in T}(E_r(i+k,j+l))^2 \sum_{(i,j)\in T}(E_l(i,j))^2}}$$

In contrary to first two metrics, this one is maximized. So, bigger the normalized cross-correlation, better the match.

Only left to discuss is the block size. What is the best choice of parameter N? Sadly, there is no answer to this question, as this is problem specific. In general bigger N gives you poor localization near the edges, and smaller N is more sensitive to noise. It is on engineer to find the best fitting block size. If no N suffices the problem, one may try to implement **adaptive block size algorithm** which uses multiple block sizes and chooses the one which gives the most similar one. It is worth noting that due to different window sizes it is important to use some kind of normalized similarity metric in order to fairly compare two block of different sizes. In the figure below you can take a look at some characteristical situations.



(a) Poor localization



(b) Noise sensitivity

Figure 3.4: Block matching characteristical situations

There is no general rule which method to use when trying to find corresponding pixels. Each problem, and circumstances dictate which similarity metric is best fitting for the solution. However, most used algorithm do implement some of the block matching techniques.

Local matching algorithms

It may not be fair not to put these algorithms in the same category as block matching algorithms, since they utilize the same technique for finding similarity be-

tween pixels. However, they are separated since they do not search the same region. Namely, these algorithm try to incorporate imminent error during production of the simple stereo system. This means that sometimes best matching pixel can not be found in the same row as pixel that we try to match. Sometimes, best matching pixel in the right image is located a row or two above or below the expected. So we can try to search more than one row, especially rows above and below $v_r = v_l$. This indeed increases complexity and slows the algorithm down.

Other ways of battling this problem may be using some real local algorithms from spectrum of artificial intelligence. These include applications of simulated annealing or even genetic algorithm as a way to find best matching pixel in smaller local area where we expect corresponding pixel to occur.

Neural networks matching algorithms

Convolutional neural networks are a big part of computer vision in modern times. Graphical processing units are getting faster, and we are able to utilize these kind of algorithms to estimate depth, sometimes even more precisely than using conventional algorithms. Using CNN we even may estimate depth using monocular vision, which were not part of this text, but is worth mentioning. We can also use CNN to enhance result of conventional stereo vision algorithms or find best matches in stereo pair. Problem lies in the fact that neural networks have to be trained on large data sets, and they are often hard to generate. Also it is important that CNN algorithm is robust enough in order to cover wide range of applications. This is also achieved through diverse data set. To conclude, neural networks could be used to solve this issue, however, one should be careful.

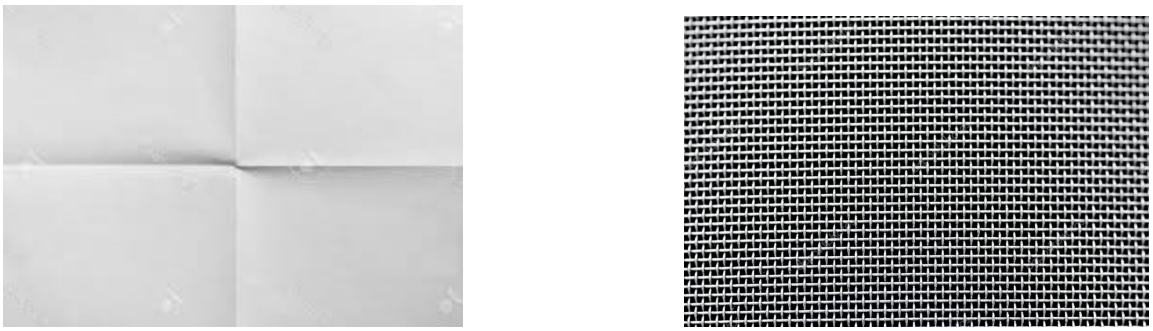
3.1.2 Simple stereo problems

Problems in simple stereo depth estimation are simply all situations where we fail to find the best match between left and right image. Additionally, there are situations where algorithm is fooled to believe that it has found perfect match, whereas it has not. Some of these situations are listed below. At the end of this chapter we will discuss briefly every one of them.

- Lack of texture
- Repetitive texture
- Foreshortening

Repetetive and lack of texture

We can merge these two situations into one. The problem with both of these situations is the same. Algorithm is not able to find the best match in any of them. Take into consideration textureless image in the figure below. One can image that trying to find the best match in any of the rows may produce many good matches. This can easily fool our algorithm into thinking it found a solution. Similarly, in images where we can find repetitive texture such as depicted below, we would have the same problem. In these two characteristical types of situations we would not be able to accurately calculate depth.



(a) Textureless image

(b) Repetitive texture

Figure 3.5: Block matching problematic situations

Foreshortening

This phenomenon very useful in art may come as quite unwanted and cause serious problems when depth estimation is considered. Problem lies in the fact that since cameras are translated, same area in the scene is not projected onto the same area in left and right image plane. In the figure show below, area in the right image would be smaller in comparison to the area in the left image. Considering we can not use one pixel as a similarity metric(it does not carry enough information), we have to use block of pixels. Now, since area projected is not the same, blocks would not correspond to exactly the same parts of the scene. They would be warped or distorted and this can could miss-matches. In order to battle foreshortening we can use warping techniques in order to try and make areas projected on both images vastly the same.

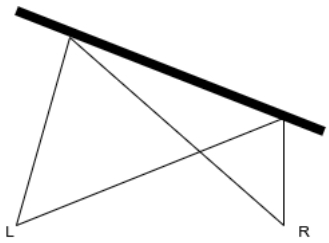


Figure 3.6: Foreshortening

3.2 Estimating depth using uncalibrated stereo

Discussing simple stereo in the section above, we have come to the conclusion that simple stereo lacks generality. It is very specific in its construction, which is definitely a flaw. We can not always gurantee that prefect simple stereo system is available to use. Sometimes we just want to use two simple images on the same objects, even taken with different cameras, just from another angle, to estimate depth. Uncalibrated stereo system gives us just that opportunity. But what is the uncalibrated part about this? From what is already said, reader may have guessed that, in contrary to simple stereo, we do not know orientation of one camera in relation to the other camera. We have to find it, or in other words calibrate the stereo system.

It is important to note that we expect that camera calibration matrices are known for both cameras, since we are going to use them during calibration process. This is not an irrational thing to ask for since matrix **K** is filled with internal camera parameters and these parameters are usually known to us. They are either given by the manufacturer or if image is found online, they are stored in the metadata. So we can consider that we are familiar with both camera parameters, without loss of generality.

This section will focus on step by step approach of calibrating stereo pair, and then estimating depth. Before we proceed into greater detail, take a look at an example of image pair that can be used to extract depth.



(a) Left camera image



(b) Right camera image

Figure 3.7: Example of uncalibrated stereo image pair (School of Electrical Engineering University of Belgrade)

3.2.1 Stereo calibration and Fundamental matrix

As it is mentioned in introduction to this section, calibrating uncalibrated stereo is the first step on the way of finding depth using any two images of one scene. We have already discussed that we expect to know camera parameters, or intrinsic parameters of both cameras. Problem of calibrating stereo pair may be understood as

a problem of finding extrinsic parameters, where we have put world coordinate system in the origin of right camera coordinate frame. This way we can find translation vector $\mathbf{t}$ and rotation matrix $\mathbf{R}$ of right camera coordinate frame in relation to left camera coordinate frame, or extrinsic parameters of the left camera. If we menage to find these parameters, we can easily triangulate depth using epipolar geometry. Take a look at the figure below which shows geometrical interpretation of unknown parameters.

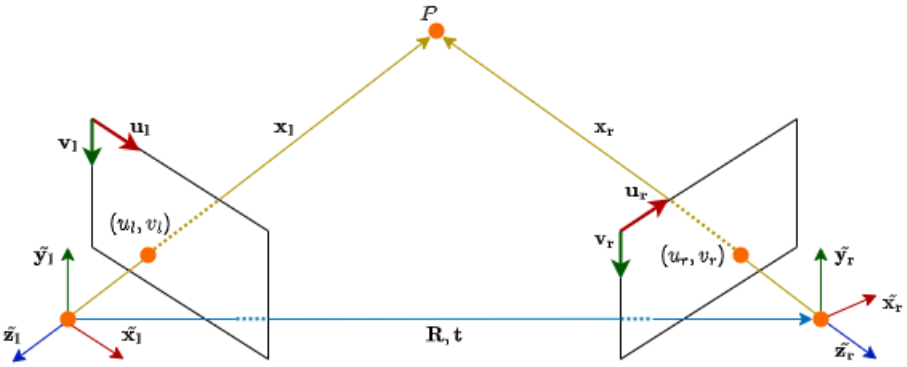


Figure 3.8: Uncalibrated stereo

Math review: Epipolar geometry

Epipolar geometry is used to help us understand how 3D points are projected onto uncalibrated stereo system, or its two camera coordinate systems for that matter. As its name suggests, it uses one "same" plane that can be formed using point in 3D and two origins. Before further discussion lets firstly take a look at the geometrical interpretation of the epipolar plane and its elements in the figure below. Image planes are removed for legibility.

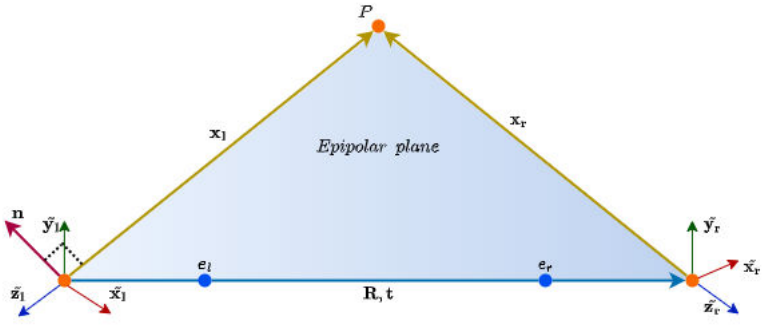


Figure 3.9: Epipolar plane

In the calibrated stereo system we know that one point in 3D projects onto the same row(horizontal line) in both images, this is not the case for the uncalibrated stereo. Purpose of this section will be to derive where projection points are located on both image plane when using uncalibrated stereo. For this purpose we introduce epipolar geometry.

Firstly lets introduce terminology. We mentioned that epipolar plane is formed by point projected and two origins O_l and O_r of two camera coordinate systems. This makes vectors $\mathbf{x_l}$ and $\mathbf{x_r}$, which represent positions of point P in two coordinate systems, lay in the epipolar plane. Vector $\mathbf{t}$ and matrix $\mathbf{R}$ are translation vector and rotation matrix of right image coordinate system in relation to the left image coordinate system, respectively. There are two new variable used, these are **epipoles** and **epipolar lines**. Epipoles are points wherewhere translation vector pierces two image planes. In the image they are labeled as e_l and e_r for the left and right image plane. Epipolar line is line where epipolar plane intersects two image planes. These two lines are not shown on the image, but they are the lines that we expect point P to be projected. For example, we know we can find a direction of a vector $\mathbf{x_l}$, or $\mathbf{x_r}$ for that matter, using homogeneous coordinates, but we can not find the length of it. In other words, we do not know how far away point it. All of this was discussed at length in chapter before. So lets imagine vector $\mathbf{x_l}$ being infinitely long, and take a tip of the vector of $\mathbf{x_r}$ and swoop it along vector $\mathbf{x_l}$. This swooping would make a plane we know as epipolar plane, and every vector $\mathbf{x_r}$ is intersecting right image plane in a point. If we concatenate these points, we have epipolar line for the right image. This holds true for both directions. We are typically interested in the right right epipolar line since we calculate depth for every point in the left image. So the algorithm would go as follows, for every pixel on the left find the epipolar line on the right, then find corresponding pixel on the right along that line and calculate depth.

But how do we find epipolar lines? We use something that is called epipolar constraint, which is no more that a plane equation that suffices epipolar plane. For every point P we have different epipolar plane. For that one epipolar plane we could use vectors $\mathbf{x_l}$ and $\mathbf{t}$ to form a plane normal. This is shown on the figure above, and the normal would be calculated using expressison:

$$\mathbf{n} = \mathbf{t} \times \mathbf{x_l}$$

And since normal $\mathbf{n}$ and vector $\mathbf{x_l}$ are perpendicular, it hold for them:

$$\mathbf{x_l} \cdot (\mathbf{t} \times \mathbf{x_l}) = 0$$

This equation is called epipolar constraint. Note that we are using $\mathbf{x_l}$ since in reality we will calculate depth on every scene point of the left image. We will use this equation to find epipolar lines and to calibrate stereo system.

Using epipolar geometry and epipolar constraint we can now calibrate the stereo pair. In the figure below we have combined epipolar plane with uncalibrated stereo pair into one image so we can derive calibration parameters more easily. There we can see all variables mentioned in the overview. Such as, epipolar lines, epipoles, translation vector, rotation matrix, image plane etc.

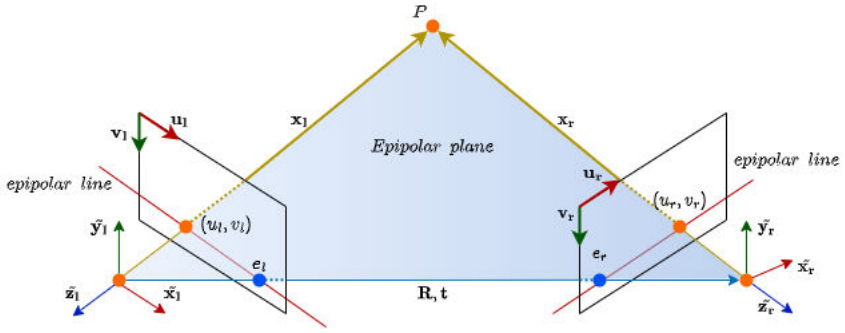


Figure 3.10: Epipolar plane

We can unitlize cross product definition in matrix form to write epipolar constraint $\mathbf{x_l} \cdot (\mathbf{t} \times \mathbf{x_l}) = 0$ in matrix from. This definition is:

$$\mathbf{t} \times \mathbf{x_l} = \begin{bmatrix} t_y z_l - t_z y_l \\ t_z x_l - t_x y_l \\ t_x y_l - t_y x_l \end{bmatrix} = \begin{bmatrix} 0 & -tz & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \begin{bmatrix} x_l \\ y_l \\ z_l \end{bmatrix}$$

Considering this epipolar constraint in matrix form becomes:

$$\mathbf{x_l} \cdot (\mathbf{t} \times \mathbf{x_l}) = \begin{bmatrix} x_l & y_l & z_l \end{bmatrix} \begin{bmatrix} 0 & -tz & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \begin{bmatrix} x_l \\ y_l \\ z_l \end{bmatrix} = 0$$

Also, we know that for two position vectors $\mathbf{x_l}$ and $\mathbf{x_r}$ and two calibration parameters matrix $\mathbf{R}$ and vector $\mathbf{t}$ holds true:

$$\begin{bmatrix} x_l \\ y_l \\ z_l \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

Now lets combine two equations:

$$\begin{bmatrix} x_l & y_l & z_l \end{bmatrix} \left(\begin{bmatrix} 0 & -tz & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} + \begin{bmatrix} 0 & -tz & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \right) = 0$$

We can remove some of the factors within this expression. For example, right addend within parentheses is cross-product of vector $\mathbf{t}$ with itself(look at the cross-product definition above), which makes this addend equal to zero. This leavs us with:

$$\begin{bmatrix} x_l & y_l & z_l \end{bmatrix} \begin{bmatrix} 0 & -tz & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} = 0$$

Here we can extract some characteristical matrices such as:

$$\mathbf{T_x} = \begin{bmatrix} 0 & -tz & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \quad and \quad \mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

These matrices are called translation matrix and rotation matrix respectively, and they are result of a calibration process. Note that translation matrix is filled with elements of translation vector, meaning that we can extract translation vector knowing this matrix. In literature product of these two matrices is called essential matrix. It can be calculated as:

$$\mathbf{E} = \mathbf{T_x R} = \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix}$$

Similarly to projection matrix, essential matrix can be decoupled to rotation and translation matrices using singular value decomposition. This is due to the fact that translation matrix is skew symmetrical and rotation matrix is orthogonal(for the same reason as in the single camera calibration procedure). This makes our epipolar constraint be equal to:

$$\mathbf{x_l}^T \mathbf{T_x R x_r} = \mathbf{x_l}^T \mathbf{E x_r} = 0$$

Unfortunately, we do not know vectors $\mathbf{x_l}$ and $\mathbf{x_r}$. They are coordinates of the point P in left and right camera system, hence we have to way of knowing them. However, we do know pixels where this point projects two. Using intrinsic parameters of two cameras and homogeneous coordinates we have:

$$z_l \begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix} = \begin{bmatrix} f_x^{(l)} & 0 & o_x^{(l)} \\ 0 & f_y^{(l)} & o_y^{(l)} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_l \\ y_l \\ z_l \end{bmatrix} \qquad z_r \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = \begin{bmatrix} f_x^{(r)} & 0 & o_x^{(r)} \\ 0 & f_y^{(r)} & o_y^{(r)} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix}$$

Here, matrices:

$$\mathbf{K_l} = \begin{bmatrix} f_x^{(l)} & 0 & o_x^{(l)} \\ 0 & f_y^{(l)} & o_y^{(l)} \\ 0 & 0 & 1 \end{bmatrix} \qquad \mathbf{K_r} = \begin{bmatrix} f_x^{(r)} & 0 & o_x^{(r)} \\ 0 & f_y^{(r)} & o_y^{(r)} \\ 0 & 0 & 1 \end{bmatrix}$$

are called calibration matrices, as mentioned in second chapter and we expect to know them during calibration process. It is worth noting that these matrices generally are the same. Meaning, we are deriving calibration process for most general case where we use two completely different cameras. If we plug these equations in epipolar constraint we get:

$$\begin{bmatrix} u_l & v_l & 1 \end{bmatrix} z_l \ {\mathbf{K_l}}^{-1} \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} z_r \ {\mathbf{K_r}}^{-1} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0$$

Now we know every parameter excluding z coordinates in relation to left and right camera. However, it pans out we do not need them. They surely can not be zero, so we can remove them from the equation and get:

$$\begin{bmatrix} u_l & v_l & 1 \end{bmatrix} {\mathbf{K_l}}^{-1} \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} {\mathbf{K_r}}^{-1} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0$$

Middle part of this equation is often rewriten into another matrix called fundamental matrix.

$$\mathbf{F} = \mathbf{K_l}^{-1} \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} \mathbf{K_r}^{-1} = \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix}$$

Similarly to projection matrix, and due to same reasons, fundamental matrix can be calculated only to a scale. Also, knowing fundamental matrix and calibration matrices we can easily calculate essential matrix and then decouple it into wanted calibration parameters, i.e. vector $\mathbf{t}$ and matrix $\mathbf{R}$. How we calculate, or estimate, this matrix will be a subject of next subsection.

3.2.2 Estimating fundamental matrix

Finding elements of the fundamental matrix $\mathbf{F}$ is a procedure quite similar to a procedure of finding projection matrix in single camera calibration. We will use some points in two image, that correspond to the same feature. Extract their coordinates and use them to solve some linear equations and extract fundamental matrix elements. So the first step is to find some pixels pairs, to be exact 8 pairs. We will discuss later why we need just8 pixel pairs, for now that information is enough. Pixel can we hand-picked, simply by clicking on the image, and using some algorithm that find similar features. Most common algorithm for this is SWIFT. Lets assume we have picked 8 points in the images of School of Electrical engineering in Belgrade, as shown in the figure below.



(a) Left camera image features



(b) Right camera image features

Figure 3.11: Example of pixel pairs for calibration

These points in left and right image are respectively(n is equal to 8 in our case, generally it can be any number of parameters more than 8):

$$(u_l^{(1)}, u_l^{(1)}) \quad \dots \quad (u_l^{(n)}, u_l^{(n)})$$

$$(u_r^{(1)}, u_r^{(1)}) \quad \dots \quad (u_r^{(n)}, u_r^{(n)})$$

Pixels or features that we match are usually some corner or other very characteristical points. Especially when we are using some kind of automated approach to extracting pixels, such as SWIFT. Simply, corners and edges are very informative, hence, very easy to detect. But ho do we use these features? Lets rewrite $\mathbf{u_l}^T \cdot \mathbf{F} \cdot \mathbf{u_r} = 0$ as linear equation:

$$(f_{11}u_r^{(i)} + f_{12}v_r^{(i)} + f_{13})u_l^{(i)} + (f_{21}u_r^{(i)} + f_{22}v_r^{(i)} + f_{23})v_l^{(i)} + f_{31}u_r^{(i)} + f_{32}v_r^{(i)} + f_{33} = 0$$

In matrix form, for all points picked, this becomes:

$$\begin{bmatrix} u_r^{(1)}u_l^{(1)} & v_r^{(1)}u_l^{(1)} & u_l^{(1)} & u_r^{(1)}v_l^{(1)} & v_r^{(1)}v_l^{(1)} & v_l^{(1)} & u_r^{(1)} & v_r^{(1)} & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_r^{(i)}u_l^{(i)} & v_r^{(i)}u_l^{(i)} & u_l^{(i)} & u_r^{(i)}v_l^{(i)} & v_r^{(i)}v_l^{(i)} & v_l^{(i)} & u_r^{(i)} & v_r^{(i)} & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_r^{(n)}u_l^{(n)} & v_r^{(n)}u_l^{(n)} & u_l^{(n)} & u_r^{(n)}v_l^{(n)} & v_r^{(n)}v_l^{(n)} & v_l^{(n)} & u_r^{(n)} & v_r^{(n)} & 1 \end{bmatrix} \begin{bmatrix} f_{11} \\ f_{12} \\ f_{13} \\ f_{21} \\ f_{22} \\ f_{23} \\ f_{31} \\ f_{32} \\ f_{33} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$\mathbf{A}_{n\times 9} \cdot \mathbf{f}_{9\times 1} = \mathbf{0}_{n\times 1}$$

As it was previously discussed, fundamental matrix and projection matrix can not be calculated exactly, only up to a scale. This is due to the nature of homogeneous coordinates. So we have to use some kind of constraint in order to battle this. Similarly to procedure of estimating projection matrix, we will calculate $\mathbf{f}$ such that $\|\mathbf{f}\|^2 = 1$. We solve this problem using constrained least square method, given by the:

$$min_{\mathbf{f}}(\|\mathbf{A} \cdot \mathbf{f}\|^2) \quad while \quad \|\mathbf{f}\|^2 = 1$$

Solving this for $\mathbf{f}$, and rearranging, gives us our fundamental matrix $\mathbf{F}$. From there we easily calculate essential matrix using:

$$\mathbf{E} = \mathbf{K_l}^T \cdot \mathbf{F} \cdot \mathbf{K_r} = \mathbf{T_x} \cdot \mathbf{R}$$

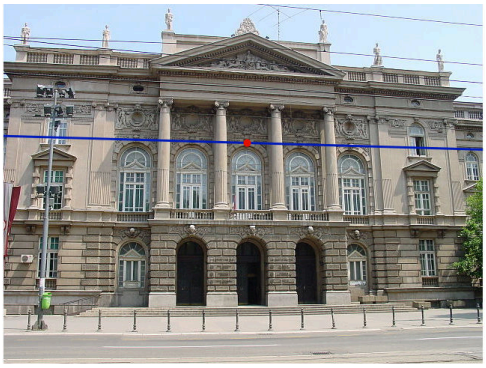
From there, it is only left to decouple $\mathbf{E}$ using singular value decomposition and extract rotation and translation of two cameras. From then on, our uncalibrated stereo system is fully calibrated, and we can proceed to depth estimation.

3.2.3 Finding correspondences

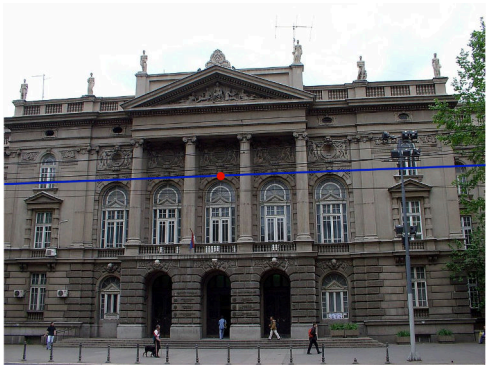
We have seen that using simple stereo, we do not have to search whole right image to find corresponding pixel in order to perform depth estimation. Since corresponding pixels are surely on the same horizontal line, simple stereo search is a one dimensional problem. Can we achieve this using uncalibrated stereo too? It turns out we can. As it was discussed before, when mentioning epipolar geometry, we came to a conclusion that every point that we projected has its own epipolar

plane. Intersections of that plane with right and left image planes are epipolar lines. This means that projected point must project somewhere on these lines.

Using typical approach to finding correspondences, we would take every pixel on the left image, and search for its pair in the right image. As it was explained, uncalibrated stereo can achieve one dimensional search, since it is guaranteed that projections must lie on both epipolar lines. In the figure below there is an example of this. We have a point in left image(red dot) and a perfect match on the right image(also a red dot). Lines colored in blue are epipolar lines for both images. We usually do not use the left epipolar line since we take a point on the left image, and search the line in the right image. Of course, this can be done both ways, this is just usual approach. Topic of this subsection will be how to find that line equation, using solely calibration parameters and pixel on the left image.



(a) Left camera point projection and its epipolar line



(b) Right camera point projection and its epipolar line

Figure 3.12: Example of point projections and its corresponding epipolar lines

This process is rather simple. Consider equation that we deriven when calibrating the camera. Because of epipolar constraint we know that it holds true for two exact matches on two images:

$$\begin{bmatrix} u_l & v_l & 1 \end{bmatrix} \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0$$

If we rewrite this into linear equation we have:

$$(f_{11}u_r + f_{12}v_r + f_{13})u_l + (f_{21}u_r + f_{22}v_r + f_{23})v_l + f_{31}u_r + f_{32}v_r + f_{33} = 0 \quad or$$

$$(f_{11}u_l + f_{21}v_l + f_{31})u_r + (f_{12}u_l + f_{22}v_l + f_{32})v_r + f_{13}u_l + f_{23}v_l + f_{33} = 0$$

Knowing all parameter f_{ij} and left image pixel coordinates (u_l, v_l) we can easily calculate linear equation of the epipolar line along which we search correspondences on the right image. This equation is given by:

$$au_r + bv_r + c = 0 \quad where$$
$$a = f_{11}u_l + f_{21}v_l + f_{31}$$
$$b = f_{12}u_l + f_{22}v_l + f_{32}$$
$$c = f_{13}u_l + f_{23}v_l + f_{33}$$

Finding correspondences is completely the same as in case of simple stereo, just over another line. We use the same template matching techniques and can come across same problems.

3.2.4 Estimating depth

This is the last step in estimating depth using uncalibrated stereo. In this subsection we expect to know calibration matrices of both cameras $\mathbf{K_l}$ and $\mathbf{K_l}$, calibration parameters of stereo calibration $\mathbf{R}$ and $\mathbf{t}$ and corresponding pixel coordinates on two cameras (u_l, v_l) and (u_r, v_r). With some simple algebra we should be able to calculate depth of the points in the right camera coordinates. From calibration matrices and pixel coordinates we have:

$$\begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix} = \begin{bmatrix} f_x^{(l)} & 0 & o_x^{(l)} & 0 \\ 0 & f_y^{(l)} & o_y^{(l)} & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_l \\ y_l \\ z_l \\ 1 \end{bmatrix} \qquad \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = \begin{bmatrix} f_x^{(r)} & 0 & o_x^{(r)} & 0 \\ 0 & f_y^{(r)} & o_y^{(r)} & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \\ 1 \end{bmatrix}$$

And using calibration parameters for transforming points from right camera coordinate system into left camera coordinate system, we have:

$$\begin{bmatrix} x_l \\ y_l \\ z_l \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \\ 1 \end{bmatrix}$$

Now we can rewrite these projection equations in terms of point coordinates in right camera coordinate system. Consider all coordinates are in homogeneous coordinate system. Equations become:

$$\begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix} = \begin{bmatrix} f_x^{(l)} & 0 & o_x^{(l)} & 0 \\ 0 & f_y^{(l)} & o_y^{(l)} & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = \begin{bmatrix} f_x^{(r)} & 0 & o_x^{(r)} & 0 \\ 0 & f_y^{(r)} & o_y^{(r)} & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \\ 1 \end{bmatrix}$$

Looking at the first equation, we can see a projection matrix of the left camera when world coordinate system is set into the origin of right camera coordinate system. Knowing this, right cameras extrinsic matrix is an identity matrix. This is due to the fact that right camera and world coordinate system are identical. For

this reason, matrix connecting pixel coordinates in the right camera and coordinates in the right camera coordinate system is intrinsic matrix of the right camera. This was discussed in the beginning of this section. So using these facts we can write following equations:

$$\mathbf{u_l} = \mathbf{P}\mathbf{x_r} \iff \begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix} \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \\ 1 \end{bmatrix}$$

$$\mathbf{u_r} = \mathbf{M_{int}}\mathbf{x_r} \iff \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \\ 1 \end{bmatrix}$$

In these equation only vector $\mathbf{x_r} = (x_r, y_r, y_r, 1)$ is unknown. Using that, we can subtract two equation and rewrite them so we calculate this unknown vector more easily. This matrix equation would be written as:

$$\begin{bmatrix} u_r m_{31} - m_{11} & u_r m_{32} - m_{12} & u_r m_{33} - m_{13} \\ v_r m_{31} - m_{21} & v_r m_{32} - m_{22} & v_r m_{33} - m_{23} \\ u_r p_{31} - p_{11} & u_r p_{32} - p_{12} & u_r p_{33} - p_{13} \\ v_r p_{31} - p_{21} & v_r p_{32} - p_{22} & v_r p_{33} - p_{23} \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} = \begin{bmatrix} m_{14} - m_{34} \\ m_{24} - m_{34} \\ p_{14} - p_{34} \\ p_{24} - p_{34} \end{bmatrix} \quad or$$

$$\mathbf{A}_{3\times4} \cdot \mathbf{x_r} = \mathbf{b}_{4\times1}$$

Knowing all parameters of matrices $\mathbf{A}_{3\times4}$ and $\mathbf{b}_{4\times1}$, we can calculate only unknown matrix $\mathbf{x_r}$. Since system is not square(we have more equations that unknown parameters), this can be solved easily using pseudo-inverse. This is done by multiplying both side of the equation with $\mathbf{A}_{3\times4}$. This gives:

$$\mathbf{A}^T\mathbf{A} \cdot \mathbf{x_r} = \mathbf{A}^T\mathbf{b}$$

Finally, solution of the depth estimation problem using uncalibrated stereo is given by the matrix equation below:

$$\mathbf{x_r} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{b}$$

Using calibration parameters of translation and rotation this can easily be transformed into vector $\mathbf{x_l}$ with following equation:

$$\begin{bmatrix} x_l \\ y_l \\ z_l \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \\ 1 \end{bmatrix}$$

3.2.5 Stereo image rectification

Although we can compute a line along which we can do block matching, even for uncalibrated stereo system, we would like not to. Property of a calibrated stereo that we can search along same horizontal line vastly eases matching procedure. This

hold true both for numerical and algorithmic complexity. It turns out we can achieve this property for uncalibrated stereo too, using process called rectification. We know that uncalibrated stereo system in general has two camera coordinate system rotated and translated in relation to each other. This is different with simple stereo, since simple stereo has only horizontal translation along x axis. Rectification is procedure that compensates for this rotation and translation. Its ultimate goal is to make left and right stereo images only horizontally shifted, so we can apply same block matching algorithms as we did with simple stereo. However, this comes with a cost. Rectified images are usually smaller that original images due to cropping. Namely, in order to achieve horizontal matching, we have to crop some part of the images. It is also worth mentioning that rectification and undistorting of an images are often done simultaneously. The reason being that both transformations are mapping pixels from one place to another, meaning that it is really effective to do them simultaneously. We will give real life example of this procedure that compensates for planar translation of the camera system with no rotation in the next chapter, but before take a look at a brief example in the figure below.

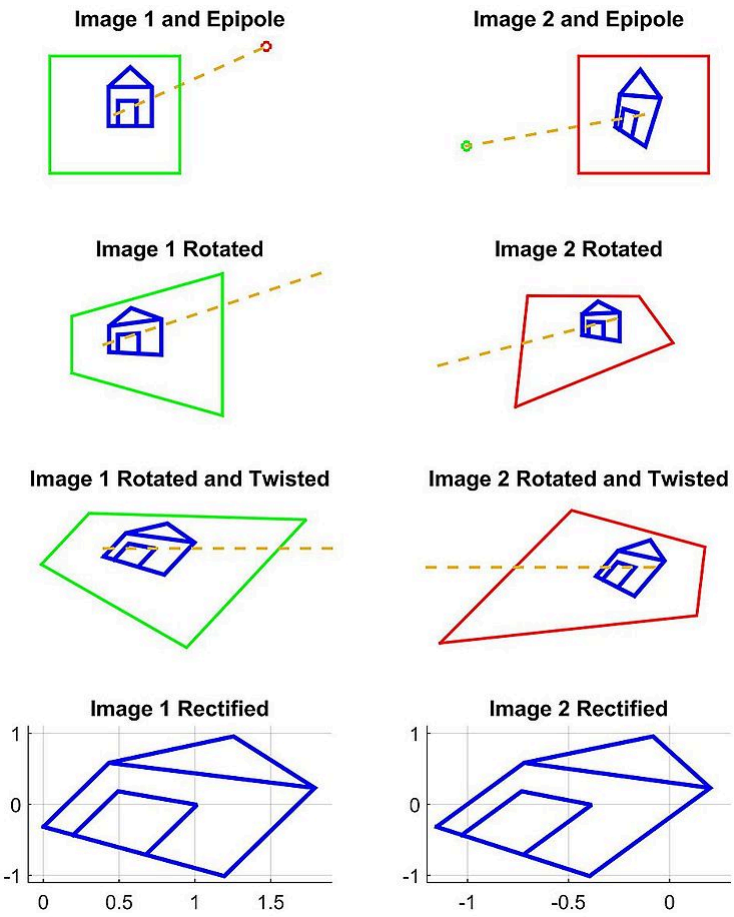


Figure 3.13: Stereo rectification

Chapter 4

Real time stereo vision depth estimation

This is the last chapter of this text. In it we will try to implement everything we learned so far and program one simple depth estimation algorithm, showing how steps of calibration and estimation are done in real world projects. We will use simple stereo system since this camera setup was available to use. However, code written can be applied to uncalibrated stereo depth estimation, considering that user implements some minor changes to it.

Algorithm in question is implementation of real time video processing, where we will calculate depth-map for every frame varying resolution and frame rate. This could be used in applications of robotics and navigation. Firstly we will discuss hardware used, and the we will proceed to algorithm and its results. What problems one may encounter when writing code and how to solve some of them.

4.1 Required hardware

For purpose of estimating depth one need a camera setup and a processing unit. When discussing cameras we can use uncalibrated or calibrated stereo pair. On the other hand, processing may be done on the personal computer or some micro-processing unit strong enough to deliver computer power to handle the algorithm. Both of these hardware components will be discussed in the following text. Including which specific components are chosen and reasoning behind. Choice of hardware components is really important especially when considering system as a whole. For example, micro-processing unit must be powerful enough to carry out computing not just for the camera, but additionally control of some motors of a robot/drone or else.

IMX219-83 Stereo Camera

As mentioned at length in this text, depth estimation can we done using simple and uncalibrated stereo. We will implement this algorithm on simple stereo system. For this purpose IMX219-83 stereo pair with two SONY image sensors with high resolution. high resolution is important so we could easily vary some characteristical resolutions and compare algorithm performance and see processing units limit. This camera system also possesses accelerometer, gyroscope and magnetometer sensors,

however, they are not of interest to us for this purpose. They could be useful when algorithm is implemented on real life robotic system. Image of the camera can be seen in the figure below.



Figure 4.1: Stereo camera pair

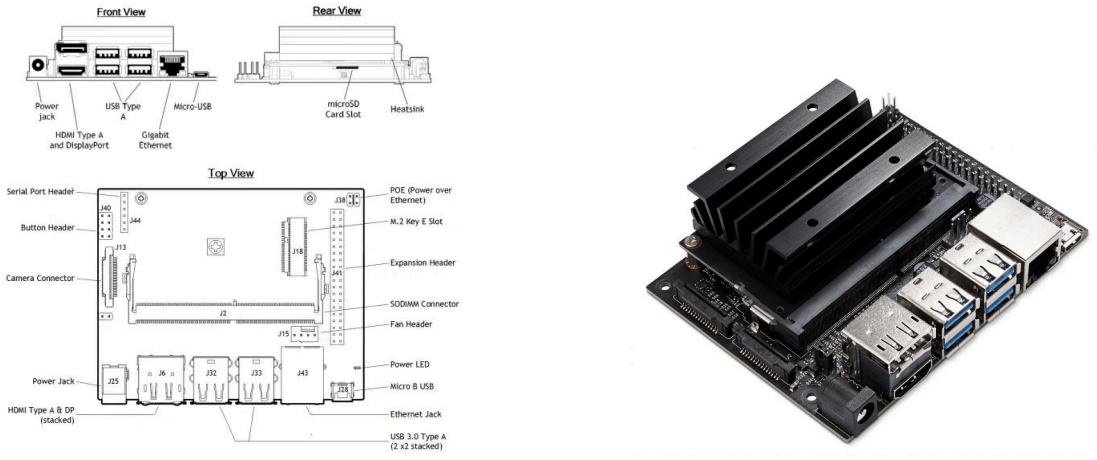
Some important parameters of the camera from the website of the manufacturer could be seen in the list below. More parameters are available on the website which is stated in the bibliography.

- 8 megapixels
- Sensor: Sony IMX219
- Resolution: 3280 × 2464 (per camera)
- Lens specifications:
  - CMOS size: 1/4inch
  - Focal Length: 2.6mm
  - Angle of View: 83/73/50 degree (diagonal/horizontal/vertical)
  - Distortion: $< 1\%$
  - Baseline Length: 60mm

Jetson Nano Developer Kit

When considering which micro-processing unit to choose, there are only two widely-spread units. These are Jetson Nano from Nvidia and Rasberry Pi. Both are quite common, can be bought easily and have a really good documentation. This is important duo to debugging purposes or even for new learners. Generally Rasberry Pi is cheaper of the two, however, it has less computational power. Jetson Nano is micro-processing unit or even micro-computer that is specialized for image processing and AI work such as convolutional neural networks. For this reason, and for further upgradeability, we chosen Nvdia Jetson Nano as a platform to perform this task of video processing. We did not consider PC, as intended use for this system

was robotics and navigation, so we nedded an embedded platform. Jetson Nano has high selection of ports available, such as USB, Ethernet, HDMI and others. Also internet accessibility is available through Ethernet port or WIFI module. Cooler is not included in the package, but is recommended due to easy overheating during heavy processing. Image of the platform and ports available are shown in the figure below.



(a) Available ports (b) Platform

Figure 4.2: Nvidia™ Jetson™ Nano

Platform operates on Linux based operating system provided on Nvidia website, and can be downloaded freely and installed. Extensive documentation about hardware and operating system can also be found on the website, along with examples. Link can be located in the bibliography. Note that this is the lowest performing platform Nvidia offers. For more upscale applications, there are much more powerful offers from the manufacturer. This is key advantage of this platform over the Raspberry Pi.

4.2 Algorithm implementation

Procedure of finding a depth map, or disparity for that matter, is basically the same as following steps that are in detail explained in chapter before this. We will do a procedure of calibrating cameras, stereo system and else as we are doing an uncalibrated stereo depth estimation. We will see what result this procedure yields and whether there is such thing as calibrated stereo in real life. Firstly, lets briefly overview algorithm of uncalibrated stereo depth estimation and then unwrap every step with some real life examples with explanation how to apply this algorithm on video depth estimation. Every material used in every step of the way along with codes is provided in appendix of this text. This way reader can try out everything we will explain in detail later and implement algorithm of its own. Additionally, for implementation of every step we used integrated functions of *openCV*. This

package offers great optimisation which we need in order to carry out real time video processing. These functions usually follow derived equation of chapters before, so we do not have solve them manually.

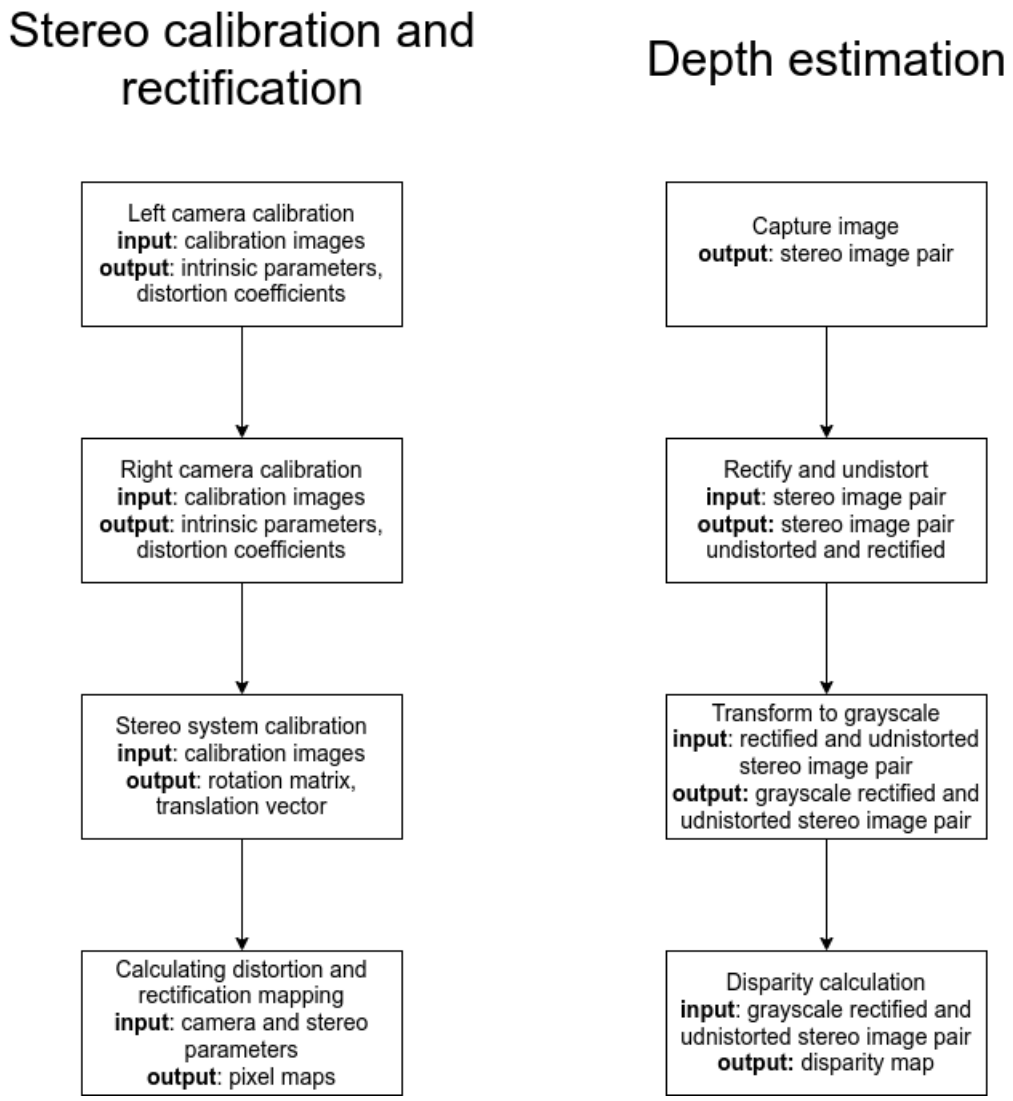
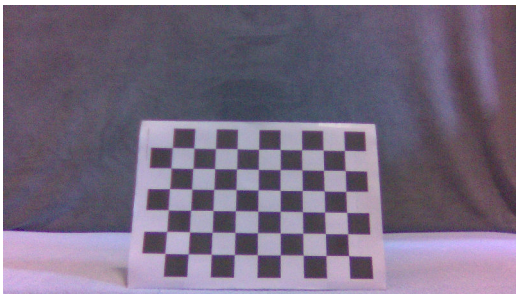


Figure 4.3: General algorithm

4.2.1 Video and image acquisition

Even though not mentioned, first step of the way is, of course, capturing images for calibration and for algorithm testing. We will keep this section short, as it is not our primary concern, however, there are some points for noting. First one video that is captured for testing purposes has to be synchronized, without synchronization depth estimation is not possible. This hold true for calibration images that are used for calibrating stereo. Single camera calibration images do not have to be in

sync. However, if we wish to use same images for calibrating stereo and calibrating individual cameras we have to ensure synchronization. This can be done either by hold the scene still when capturing left and right images, or make image reading synchronization with hardware. We will discuss more properties of good calibration images and good test pair later. But before, here are some examples of images used in the project.



(a) Calibration pattern image



(b) Test image

Figure 4.4: Captured images example

4.2.2 Camera calibration

In this part of this section we will discuss how camera calibration process is done in real life. As mentioned, we will pretend we used uncalibrated stereo system and use result to check whether this was a shrewd decision. Calibration can generally be divided into two parts. First part is calibrating cameras individually, and second is calibrating cameras as a stereo pair. We will cover both of them with some examples and advice for good quality stereo calibration.

Single camera calibration

We have already shown one image of the calibration image used. One may notice that this pattern counters what we used for deriving calibration procedure. Reason for using checkerboard pattern instead of cube is practical, it is much easier to locate internal corners, or characteristical point with planar calibration pattern than cube pattern. In case cube patterns we would have to know on which side of the the cube corner occurred, and then calculate world coordinate points. With planar calibration pattern, same as we used, we can place xy plane in the same plan as our pattern an easily find where the world coordinates of each internal corner are. We know how many corners we expect per calibration image, since we know the shape of the checkerboard. We also know real world size of checks, however, this is not important in the process since matrices in the calibration procedure are found only up to a scale. In other words, results are the same for both using real size or normalizing to one. Also, we never use one calibration image, even though we explained that only a couple of points is needed to acquire camera parameters. This

is due to the fact that we did not include lens distortion in these equations. So for this reason, and for accuracy of the parameters calculated, we use more images. In our case we used thirty images, and later we will see did this help us achieve better results.

So the first step of calibration is to acquire all available points from all the patterns available and pair them with their corresponding world coordinates. Finding calibration pattern corners is done on grey-scale calibration image using template matching techniques. This is done with *openCV* function:

$$cv.findChessboardCorners(image, patternSize, flags)$$

Function generally find corners easily since they are quite characteristical, or are hardly found in real world images, so surrounding of the pattern do not hamper the template matching. Algorithm also has exact number of the corner to find given to it, which along with knowledge of pattern shape, makes this matching even easier. Here is one output of such function on our calibration images in resolution 1920×1080.

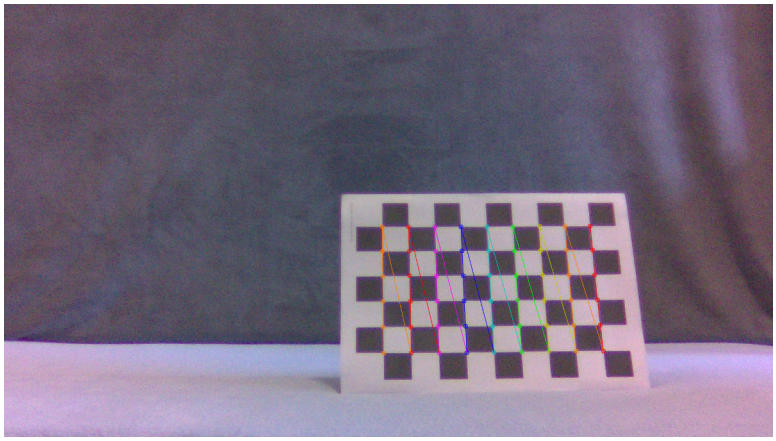


Figure 4.5: Calibration pattern corner template matching

Corner that function matched can be found in its output values along with a flag whether function found all corners or not. These pixel coordinates are paired with correspoding world coordinates and appended into an array of points. This procedure is done for each of thirty calibration images. When all images are processed, these points are sent to another *openCV* function that performs calculation of all camera parameters and distortion coefficients using same derived equations from chapter two with addition of distortion. This function is:

$$cv.calibrateCamera(worldCoordinates, imCoordinates, imSize, flags, criteria)$$

Since we apply same procedure for both camera of the stereo pair. We will take a look at results we got using this algorithm for left and right images on resolution of 640×480. So calibration matrices for each camera are:

$$\mathbf{K^l_{3x3}} \approx \begin{bmatrix} 586.5 & 0 & 336.4 \\ 0 & 782.9 & 182.0 \\ 0 & 0 & 1 \end{bmatrix} \qquad \mathbf{K^r_{3x3}} \approx \begin{bmatrix} 581.4 & 0 & 334.5 \\ 0 & 777.6 & 243.1 \\ 0 & 0 & 1 \end{bmatrix}$$

And distortion coefficients are:

$$distortion_{left} \approx [-0.16 \;\; 1.26 \;\; -0.0086 \;\; 0.00134 \;\; -2.80]$$

$$distortion_{right} \approx [-0.19 \;\; 1.95 \;\; -0.0075 \;\; 0.0003 \;\; -6.12]$$

We have taken five distortion coefficients for each camera for accuracy, even though only a couple of are enough. They are small as expected and differ a little bit for each camera since they are never exactly similar. This is also true for calibration matrices, they are quite similar but not the same. This is due to the manufacturing process and numerical inaccuracy. Lets check whether number we have in calibration matrices are what we expect. Take upper right value as an example. This is optical centre along x axis, which is 640 pixels in width. Values 336.4 and 334.5 are somewhere in the middle of this image, which is what we want.

Important value for calibration process is root mean squared error we get as an output value of camera calibration function. In general, we can tolerate this value to be up to 2 or 3, but not above. Everything below 1 is really good, and we expect this parameters to be higher for higher resolution since we need more images to calibrate higher resolution camera. Take a look at a table of our root mean squared error values we got for each resolution as a proof of good calibration process.

Resolution	(1920 x 1080)	(1280 x 720)	(640 x 480)
left camera	0.771627057333396	0.4968025328225817	0.29697521243413644
right camera	0.838242544219117	0.5033136821235785	0.2965691022235138

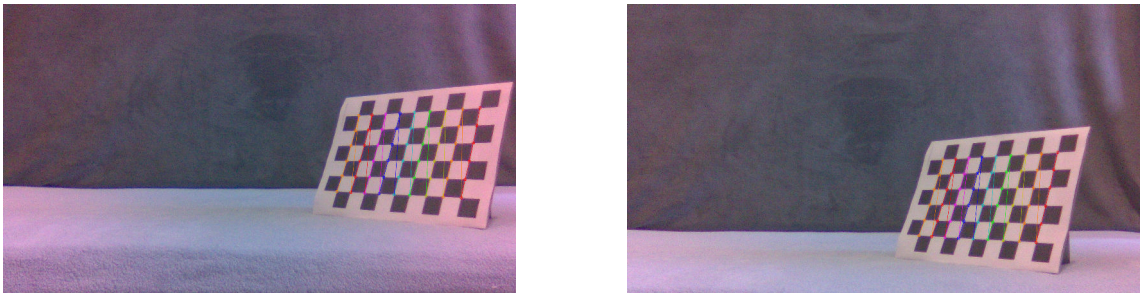
Table 4.1: RMSE single camera calibration

Stereo pair Calibration

After we obtained camera parameters for both cameras individually, we can proceed to calibrating them as a pair. As explained in chapters before, calibrating a stereo system means finding rotation matrix and translation vector which describe orientation of the right camera coordinate frame in relation to the left camera coordinate frame. Procedure is quite similar to calibrating single camera. One should firstly find some characteristical points in the calibration pattern, or any object for that matter, find world coordinates and image coordinates and then use some kind of template matching algorithm such as SIFT or RANSAC(in out implementation we use RANSAC). This template matching is used to connect corresponding points in left and right image. We will later use these pairs of image coordinates, along with world coordinate of that point on the calibration pattern to calibrate stereo.

As mentioned, first part is finding corners in the calibration pattern. We used the same pattern here as we did in calibrating single camera. Process of finding corners is nearly identical to that in calibrating cameras individually, with difference that we

do it for pairs of images. So one iteration of corner finding algorithm goes as follows: finding exact number of corner in left and right image and matching them. Then we repeat for every image in the calibration set, which is in our case thirty images. We used the same images from calibrating left and right camera since we captured them synchronized. This is very important, as explained before, since we would not have the same scene in the right and left frame. Further matching of these frames that are not in sync would make no sense. We have achieved this synchronization trough stabilizing calibration pattern when capturing left and right images. When corners are found, we would have an image pair looking as in figure below(given for resolution 1920 x 1080).



(a) Left stereo image (b) Right stereo image

Figure 4.6: Template matching between stereo pair

After finding corner for all image pairs given. We use simple *openCV* integrated function to calculate rotation and translation matrix. This function is:

$$cv.stereoCalibrate(input\ \ parameters)$$

Here with input parameters we mean both intrinsic matrices for both cameras, distortion coefficients also for both cameras, world coordinate corners, image coordinate corners in both images, image size and some additional parameters in form of criteria and flags. All these can be looked up in *openCV* documentation in greater detail. As mentioned. most important output parameters of the function are rotation matrix and translation vector. Lets take a look at parameters we got for the resolution 640 x 480.

$$\mathbf{T} \approx \begin{bmatrix} -0.06168691 \\ -0.00022198 \\ -0.00722948 \end{bmatrix} \qquad \mathbf{R} \approx \begin{bmatrix} 0.9999 & -0.0029 & -0.0004 \\ 0.0029 & 0.9999 & -0.0082 \\ 0.0005 & 0.0082 & 0.9999 \end{bmatrix}$$

Firstly it is worth mentioning that these parameters have their values in meters, so they are easily fact checked and more importantly they are roughly the same for every resolution. One can look this up the appendix. Since our camera system is a simple stereo pair, we expect that we have no rotation along any axis. Proof of this is rotation matrix we got from calibration process. Our rotation matrix is approximately identity matrix (has ones along main diagonal and zeros on other places). This means that there is no rotation between two coordinate frames.

On the other hand, translation matrix is more interesting. We expect to have translation along one axis measuring exactly $60mm$(this is our baseline of the simple stereo), we can verify this by looking at the first component of the vector. Although, take note that our value is negative. This is due to the fact that our cameras are fixed on the stand upside down. This makes no difference in calibration or later on results, but is worth mentioning. We expect other two translation component to be zero in order this to be simple stereo. However, they are not. One component of the two is actually small, this should be our z component. It is close to zero since our sensor lay on the flat PCB. This takes manufacturing errors to a minimum. On the other hand, third component is not close to zero, in fact it is approximately $7mm$, which is not small translation by any means. If we look again closely at the image of the calibration matching above, we can see that calibration pattern is definitely not translated only horizontally, but vertically too. Just by looking on the image we could see that our translation vector would have more than one non zero value.

This means our assumption to consider this a non simple stereo payed off. Trying to block match pair like this just by searching inline would yield to terrible results. We will show result of inline matching later in the text, for now it is important to note that we have uncalibrated stereo at our hands and that it is important to rectify it in order to have successfull block matching.

At last, it is time to look at the calibration error in form of root mean squared error. This parameter is returned as an output value of stereo calibration function, similarily to single camera calibration. Everything we sad before when mentioning individual calibration holds true for stereo calibration error. Shortly, everything below 1 is perfect, we can tolerate up to 2 or 3. Also, error increases with resolution with same number of calibration images. Take a look at the table below.

Resolution	(1920 x 1080)	(1280 x 720)	(640 x 480)
RMSE	0.8616381659325243	0.5556982478602257	0.32668337691798094

Table 4.2: RMSE stereo camera calibration

4.2.3 Rectification and undistortion

We proved trough calibration procedure that our stereo system is not calibrated as we thought it was. For this reason we need to rectify stereo system in order to be able to block match along the same line. Additionally we would need to do compensation for lens distortion anyways, so lets try to do these transformations together. This is possible due to the fact that both transformation are in their core a pixel mapping transformations. This means that both transformations are specified with a map of where each pixel of non rectified image should be placed in the rectified image. This property allows us to combine rectification and undistortion in one transformation, which also increases speed of our algorithm.

These maps are calculated using distortion coefficient which are responsible for compensating lens distortion, and rotation matrix combined with translation vector to compensate for warping. You mat notice how these maps do not change over the time. All parameters are fixed for a given resolution and stereo system. This means we do not have to calculate these maps over and over again. Simply, whenever we

need to use this calibrated camera with this resolution, we load these maps and use them, without need to calculate them over every iteration.

For purpose of calculating rectification and undistortion maps we use two *openCV* functions. First one is responsible for calculating rotation and projection matrices from one camera to another. Function uses intrinsic parameters of both cameras, distortion coefficients and results of stereo calibration process, i.e. rotation matrix and translation vector, along with some other flags and parameter α which we will explain later in the text. Important flag we usually set is $cv.CALIB_ZERO_DISPARITY$ flag. This tells our rectifying algorithm to rectify images so their y components are aligned. Exact signature of a function is as following:

$$cv.stereoRectify(input\ \ parameters)$$

After acquiring all these parameters we can use them to calculate maps we need. Maps are calculated with help of parameters of a single camera we calculate maps for. There are two maps for each camera, one for remaping x component, and other for remaping y component. Simply, each map is sized as an input image and each pixel x and y component is transformed using corresponding mapping. Calculation of these maps is done using following function:

$$cv.initUndistortRectifyMap(input\ \ parameters)$$

In the figure below we can see a pair of test images that are rectified and output of rectification and undistortion algorithm using maps generated by function given above.



(a) Left stereo image



(b) Right stereo image



(c) Rectified left stereo image



(d) Rectified right stereo image

Figure 4.7: Stereo rectification and undistortion

Considering that for our stereo system, as we saw, distortion caused by lenses is not large. This is due to the fact that camera pair does not have huge lenses with wide viewing angle. Also, due to the fact that our camera system is only translated in xy plane, without translation over z axis or any kind of rotation, we are able

to keep good portion of our input images. Generally, rectification and undistortion process can badly crop images and make then unusable, so one should be really careful. Responsible for amount of black parts left due to the remaping process is parameter α which is used as an input parameter for the function above. Parameter takes values from range $[-1,\ 1]$, where -1 is for maximum optimization of the black parts and 1 for no optimization. By black parts we mean an output pixels where no pixel is mapped to. We keep this parameter at zero to retain as much of input image as possible as none of the distorting effect are present at large amount.

Images that we got as a result of remaping input images to compensate for distortion and warping are cropped in relation to original images. This is something we expected. However, we can see that now images are much more horizontally aligned than before. Not perfect, but much better than before. This will help us calculate disparities much more accurately.

4.2.4 Disparity calculation

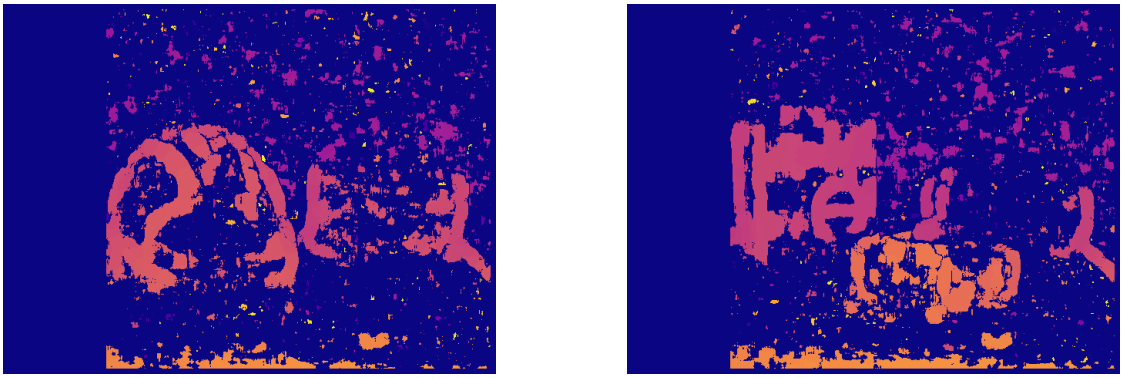
Last step of our way of estimating depth is calculating disparity map. We explained that disparity map is horizontal difference between two pixels on the left and right image which we use to calculate depth. Transforming disparity to depth is a simple process of dividing constant value $f_x \cdot b$, where b is baseline and f_x is focal length in pixels along x axis, with disparity map. It was discussed before that pixel matching can be done both with local and inline algorithms, we implemented both, so we can compare them. Before applying these disparity calculating algorithms, we do some preproccessing. This preprocessing is of course rectification and undistortion with maps we calculated. Maps do not have to be recalculated, as we explained, if we keep the resolution of the stereo constant. This is particularly important for video estimation which we will see at the end of this chapter. For now it is important to calculated these undistorted and rectified new images by remaping, then turn them greyscale. This greyscale pair is forwarded to our algorithms which does disparity calculation for us. We will cover both of them briefly and show some results on test scene shown above.

Inline block matching

Since we rectified or images, we expect inline matching algorithm to work properly. However looking and the figure below, this is not the case. We displayed two different test scenes and we will discuss the results. Algorithm of inline block matching is simple, we give it size of the block to match and number of disparities to calculate. Often it is not necessary to search the whole width of the image. In our case we search exactly the fifth of it. Block matching gives us the best matching on that number of pixels which is then translated to disparity easily.

Negative sides of this algorithm are that it is too simple. Firstly we search only horizontally, expecting that rectification is done perfectly, which is never the case. Even worse if we expected this setup to be simple stereo. We would find nearly no good matches. Looking at these images we see a lot of noise and missmatches. The reason being is that the scenes are quite textureless. For example, protective hat on scene one is completely white, so matching algorithm find best match near the original pixel. This gives us blue values on our disparity maps, which means that disparity is zero. This is the case for all other textureless surfaces on the scene,

which are many. Even though outlines of the object are matched right, due to the fact that edges are characteristical, these results are rather unusable. Obviously we would have to search more than just one horizontal line, which leads us to local approach of this algorithm. We will discuss this in detail in the next subsection.



(a) Test scene 1 (b) Test scene 2

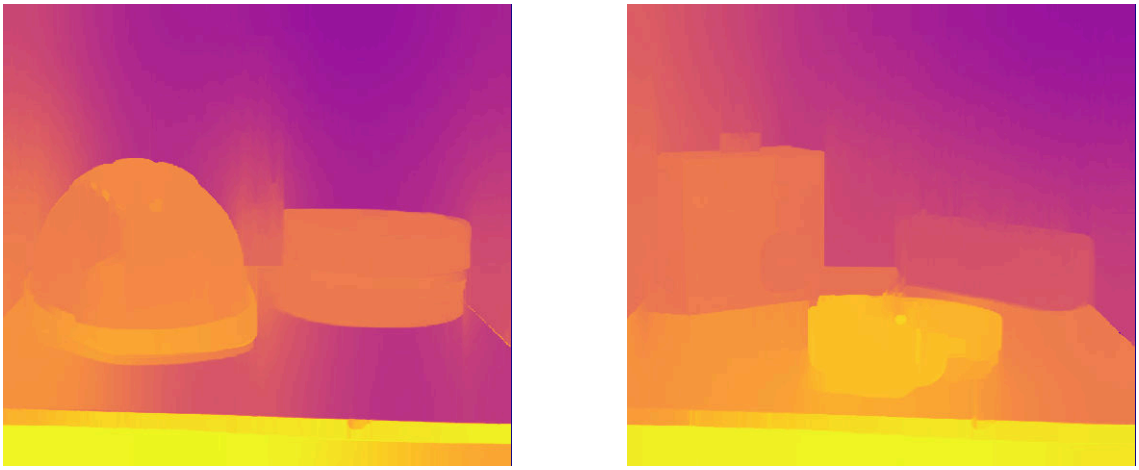
Figure 4.8: Inline matching examples

Additionally, something worth noting is the left part of these disparity maps are filled with zero columns. This is due to the fact that images are horizontally shifted for approximately $60mm$. We left these blue parts in these images on purpose here, however, we will crop them in the final algorithm.

Local block matching

We have seen that matching just horizontally does not yield results that suffice our needs. This means we will have to search elsewhere on the image. But where? Answer to this is local matching or in *openCV* terms, semi global block matching algorithm. This algorithm simply extends the search along more horizontal lines in the image, trying to find better matches. This compensates for imperfect rectification process, and makes results increasingly better. Also we used weighted least squares filter in order to smoother disparity calculation and remove outliers. This filter uses stochastic properties of the region and smooths out an input disparity map. We can control how aggresive filtering is by setting parameter λ. Lambda parameters tell us how good filter retains edges of the original image. Preset value is 8000, higher the value more disparity edges adhere to original image edges. We used lambda equal to 30000, as it gave the best results. Another relevant parameter is sigmaColor. This defines how sensitive the filtering process is to source image edges. Large values can lead to disparity leakage through low-contrast edges. Small values can make the filter too sensitive to noise and textures in the source image. Typical values range from 0.8 to 2.0. Filtering also uses right to left disparity map, not just left to right, to validate the results of filtering. Other parameters include parameters of block matching algorithm, these include accuracy, block size, number of disparities calculated and else. These have to be set according to the situation

or resolution. For example, bigger resolution need bigger block size. Also better accuracy slows down disparity calculation which can be problematic when processing time is of interest.

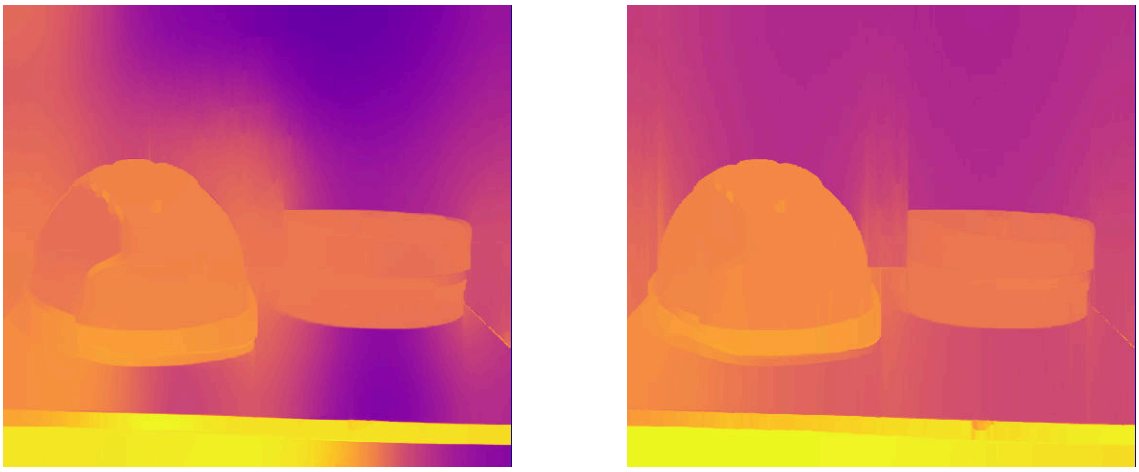


(a) Test scene 1 (b) Test scene 2

Figure 4.9: Local matching examples

Reader can test out this algorithm by setting up different parameters for different scenarios by altering code given in the appendix. We have displayed pair of results for two different scenes at resolution of 640 x 480. This resolution is considered as it will be used for real time depth estimation, so these result are most relevant. Looking at the results we see that good segmentation is achieved and more importantly closer objects have higher disparity values, opposing to objects that are farther away. This technique also suffer from the problem of lack of texture. this can be seen when algorithm calculates disparities of the table on the left image. This problem is, however, solved on the image on the right. Also, we see a great deal of details on the objects, such as protective hat on the left image, or in the headphones on the right image. Which is good. Note that these disparity maps are cropped to compensate for horizontal shift, as we mentioned before. It is worth noting that shadow can helps us a great deal when estimating depth, we can see this on example of white textureless hat on the left image, since it gives texture. As a matter of fact, when doing 3D reconstruction, object of interest is often imaged with different textured light in order to make matching process better.

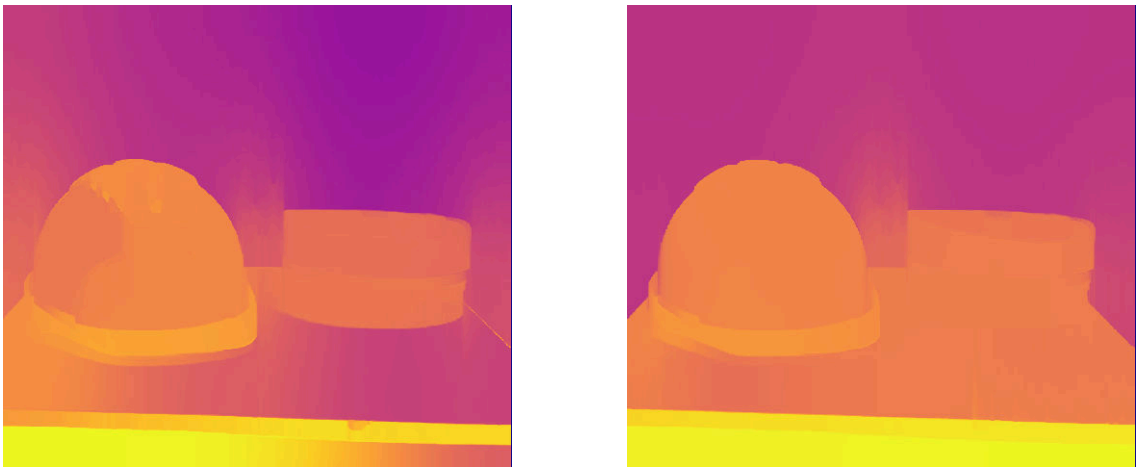
Now, what if the altered some parameters? Lets consider three parameters which are most relevant. First is parameter lambda, the same one we discussed before, next one is window size, and last is number of disparities. In the following text we will explain each one of them and give some examples. Firstly, lets discuss parameter lambda. Increasing it makes disparity map adhere more to the original image edges. Take test scene as an example in the figure given below. In this example we displayed disparities with lambda equal to 8000 and lambda equal to 80000, so ten times the difference. We do this in order to emphasize the result. With smaller lambda as expected we have disparity map that is somewhat blurred.



(a) Lambda = 8000 (b) Lambda = 80000

Figure 4.10: Parameter lambda

Next parameter we will discuss is windows size. As explained in chapter before, this parameter acts on localization and represents size of the block we use for matching. Bigger the window size, less localized edges we have and less detail. However, if we make this parameter too small, our disparity map can suffer from noise. Take a look at example below, given for window sizes 1 and 33.



(a) Window size = 1 (b) Window size = 33

Figure 4.11: Parameter window size

Bear in mind that this parameter is heavily influenced by resolution. Bigger the resolution, bigger the window size has to be for the same result. In the figure below see a two good results for image with resolution 640 x 480 and 1280 x 720 and their respective window sizes.

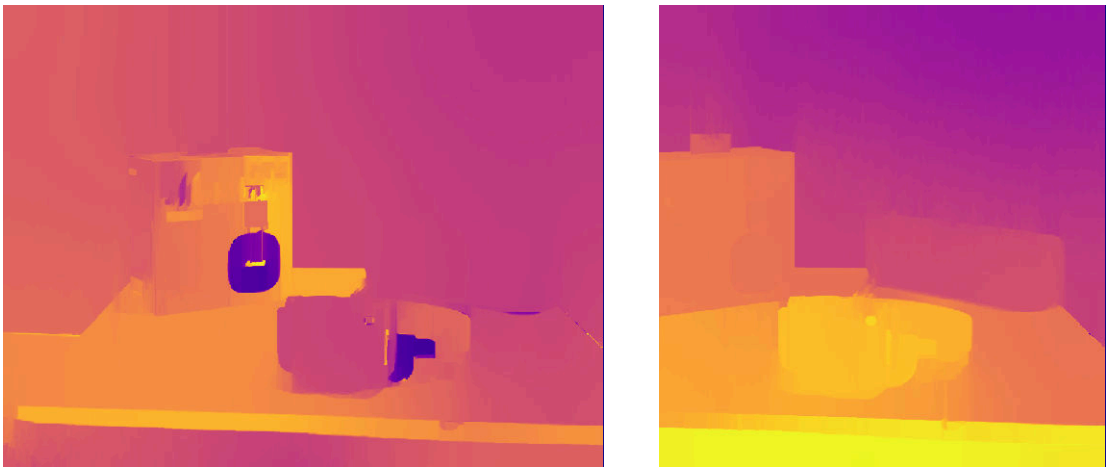


(a) 640 x 480, window size = 5

(b) 1280 x 720, window size = 11

Figure 4.12: Resolution and window size

Last, but not the least important parameter we will discuss is number of disparities. This parameters is directly in relation to precision of the disparity map. It tells us in how many values disparity can take. For example, when minimum disparity is -1, and number of disparities is 16, disparity map can take values from range [-1, 15]. In openCV implementation, this value must be divisble by 16, for faster processing. Also, this parameter directly increases processing time of the matching, so for real time application, we would like to make this parameter as small as it can be. Further reason to decrease this parameter is noise, bigger the accuracy, more the disparity image is prone to noise. Since we already showed an image with right value of number of disparities(5*16), figure below shows two extremes of setting this parameter, i.e. too low, and too high.



(a) Number of disparities = 1 * 16

(b) Number of disparities = 11 * 16

Figure 4.13: Parameter number of disparities

This parameter also alter the size, or width, of the output image. Namely, since we specify number of disparities that are possible, the bigger this parameter is, more of the image we have to search. So on the right side of the left image we will have some cropping since disparities can not be calculated for these value, due to the fact that there are not enough pixels on the right image to match.

4.3 Performance and Optimisation

In the end, lets take a look at the performance of algorithm of disparity calculation. We know that rectification maps do not have to be calculated every time we do an estimation, only once, so we will not include them when calculating time of the algorithm takes to execute. Whole process of calibrating cameras and calculating rectification maps takes only a couple of seconds. This is not much but if we did this every time, we would have no chance of any real time processing. We are also not going to include time to read stereo image pair, since this differs depending on whether we read image from local storage or directly from camera. This leaves us only with four operations. These are:

1. Remaping stereo images
2. Transforming stereo images to greyscale
3. Disparity map calculation
4. Disparity map cropping
5. (Optional) Calculating depth

Steps one and two can be rotated so we firstly do transformation of image to grayscale. This in fact can be beneficial in terms of speed since this optimisation causes rectification to be done only on one component, instead of three RGB components. Rectification maps anyway do not have three components, just one, sized at the resolution of image recitified, so the same map is applied to all three components. As we are anyway going to greyscale the image after the rectification in order to calculate disparities, we could do this before and optimise the process. Other steps stay the same, however, we will not do and optional step of depth calculation since its trivial. In the four tables below we will compare whether mentioned optimisation helps us, and how much. Also we are going to compare two platforms, Jetson Nano and PC. It may be worth mentioning that PC is specced with 11th Gen Intel Core i7-1185G7, 48GB RAM and integrated Mesa Intel Xe Graphics. We expect PC to perform better since it has much more computing power. According to tables below we will see that in general Jetson Nano is five to six times slower. Also, all codes run on PC are in Python programming language, whereas all code run on Jetson Nano is written in C++. Writing code in C++ speed up the execution as Python is an interpreter programming language which firstly interprets code to C++ and then runs it. We also expect that execution times is proportional to resolution.

Resolution	(1920 x 1080)	(1280 x 720)	(640 x 480)
Test scene 1	1876 ms	804 ms	275 ms
Test scene 2	1812 ms	728 ms	250 ms
Test scene 3	1789 ms	757 ms	249 ms

Table 4.3: Local matching performance rectification first - Jetson Nano

Resolution	(1920 x 1080)	(1280 x 720)	(640 x 480)
Test scene 1	380.1 ms	119.9 ms	43.9 ms
Test scene 2	309.2 ms	142.3 ms	41.3 ms
Test scene 3	396.7 ms	126.5 ms	42.5 ms

Table 4.4: Local matching performance rectification first - PC

Resolution	(1920 x 1080)	(1280 x 720)	(640 x 480)
Test scene 1	1842 ms	766 ms	256 ms
Test scene 2	1734 ms	717 ms	246 ms
Test scene 3	1804 ms	704 ms	237 ms

Table 4.5: Local matching performance grayscale first - Jetson Nano

Resolution	(1920 x 1080)	(1280 x 720)	(640 x 480)
Test scene 1	289.4 ms	122.8 ms	34.6 ms
Test scene 2	290.3 ms	113.4 ms	32.5 ms
Test scene 3	294.4 ms	126.0 ms	39.8 ms

Table 4.6: Local matching performance grayscale first - PC

Note that these tables are generated randomly. They are not calculated over some statistical analysis, however, they are a good representative of what we expect of execution times to be. Also, algorithm implemented for generating these tables are exactly the same, only rewritten in another programming language and run on different platform so they could be comparable. Real time application require additional optimization which we will discuss later.

Jetson Nano is far from ordinary micro-processing unit. It was developed to be able to carry out real time application such as this. But how is this possible? Looking at the tables above, Jetson should not be able to pull out more than a couple of frames per second. The answer is in specialized CUDA cores. Namely, CUDA cores are optimized graphical processing core optimzed for large scale nummerical calculation. Such as this. OpenCV library can be combined both with ordinary CPU, or if available with more competent CUDA GPU cores. Tables above are generated with CUDA optimization off.

Using CUDA optimization, Jetson Nano is capable of executing real time processing applications. Even more so if we implement some more optimisation fitting to a real time processing which will be topic of the following section.

At the end, it is worth mentioning that Jetson on full load consumpts 5W of power from a power source, or bellow 3W when idling. Power source can be both a battery or a power outlet. Typical consumption of a CPU in modern PC is around 60 to 70W. This makes PC completely unusable in applications such as mobile robotics. On the other hand, Jetson can easily operate on a battery power for extended period of time due to the much lower consumption. So being able to run programmes on embedded platfom such as Jetson is crucial.

4.4 Video depth estimation



(a) Disparity map frame 1 (b) Disparity map frame 2 (c) Disparity map frame 3

Figure 4.14: Real time video depth estimation examples

To close down this section, we will combine everything we learned to implement algorithm which estimates depth in real time. This procedure does not differ at all to ordinary depth estimation/disparity estimation on single image. In fact it is the same. We know that video is just a bunch of images played at specific rate, known as frame rate. So we simply apply depth estimation of every frame pair. Bear in mind that estimation has to be fast enough in order to be applicable in real time. For example, in we record a video in 20 frames per second, algorithm has $50ms$ to finish reading, rectifying and estimating. Looking at tables above, we see this is only possible for resolution 640 x 480 and frame rate of a little bit over 20 frames per second. For this reason it is important to implement every optimisation available to us. So it is preferable to do greyscale transformation before rectification as we discussed. Also for maximum algorithm speed, one should implement it using C++, which is significantly faster than Python. Moreover, discussed CUDA optimisation is a must in processing like this. Enabling graphical processing of Jetson Nano, reduces load from CPU and speeds up numerical calculation. Some other optimisation include altering parameters or code refactoring. Example of parameter altering is reducing window size or number of disparities. For this purpose we used window size of 3 and 3*16 disparities. Furthermore, code refactoring is important in order to reduce number of large matrix data to be passed through functions. Generally, in order for these real time algorithms to work, it is advisable to use integrated functions as they are heavily optimized opposing to user-defined functions which tend to be much slower. Another thing worth mentioning is that frames have to be synchronized, similarly to stereo calibration images. Without synchronization, estimation would not be possible. We shown some examples of frames recorder in the figure above this paragraph, as well as algorithm we implemented below this paragraph.

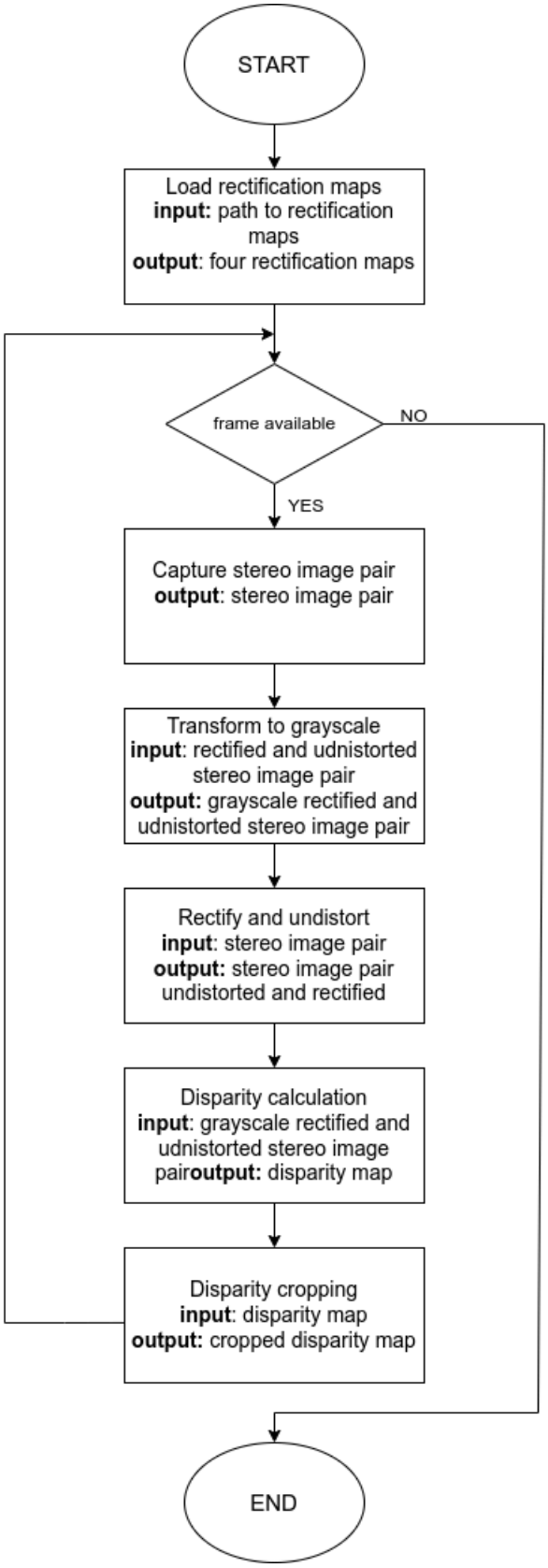


Figure 4.15: Real time video depth estimation algorithm

Chapter 5

Conclusion

Depth estimation is one of the most fundamental problems of computer vision. And one of the most important. As it was discussed we can use depth estimation for numerous purposes. Some examples are 3D modeling, or factory automation. More recent applications include detecting potholes in roads using depth-maps, or even pedestrian. Shortly, uses are extensive, covering all spheres.

This text had a purpose of introducing a reader that is not familiar with the topic. Reader that wants to immerse oneself into this important computer vision problem. It cover inextricable topics connected to depth estimation. Such as camera modeling and calibration. Intention was to completely cover all topic surrounding depth estimation, and equip reader with knowledge important to start implementing practical algorithms of depth estimation or its applications.

At the end we cover one of those applications and go trough its practical implementation using *openCV*. This application is real time video depth estimation, used for robotics, autonomous driving, sensor fusion etc. Hopefully, after reading this reader is prepared to face challenges of depth estimation in real world, and implement some more complex algorithms.

Appendix A

Additional materials

A.1 Calibration and test files:

1. Calibration images for three different resolutions: 1920x1080, 1280x720 and 640x480. Thirty checkerboard calibration images for each resolution.
2. Calibration results for three different resolutions: 1920x1080, 1280x720 and 640x480. Intrinsic matrices and distortion coefficients for both left and right camera for each resolution. Rotation matrices and translation matrices of stereo pair for each resolution.
3. Test images for three different resolutions: 1920x1080, 1280x720 and 640x480.
4. Test videos for three different resolutions: 1920x1080, 1280x720 and 640x480. Filmed with 20 frames per second.
5. Depth estimation results for all test images given for two matching algorithms implemented.
6. Calibration pattern

A.2 Codes:

1. Calibration library in Python programming language
2. Rectification library in Python programming language
3. Depth estimation library in Python programming language
4. Depth estimation single image test script in Python programming language
5. Depth estimation test video script in Python programming language
6. Depth estimation in real time script in Python programming language
7. Depth estimation single image test script in C++ programming language
8. Depth estimation in real time script in C++ programming language
9. Video and image capturing library in C++ programming language

Bibliography

[1] Berthold K. P. Horn. *Robot Vision*. 1986.

[2] Richard E. Woods Rafael C. Gonzalez. *Digital Image Processing Third Edition*. Pearson, 2008.

[3] Jean Ponce David A. Forsyth. *COMPUTER VISION: A MODERN APPROACH, 2ND EDITION*. Pearson, 2013.

[4] Richard Szeliski. *Computer Vision: Algorithms and Applications 2nd Edition*. Springer, 2022.

[5] Intel ™. *OpenCV: documentation, library, examples and basics*. URL: https://opencv.org/.

[6] MatLab ™. *MatLab ™: Camera calibration documentation*. URL: https://www.mathworks.com/help/vision/ug/camera-calibration.html.

[7] Nvidia ™. *Nvidia: Jetson family documentation*. URL: https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/.

[8] WaveShare ™. *WaveShare ™: IMX219-83 documentation*. URL: https://www.waveshare.com/wiki/IMX219-83_Stereo_Camera.

[9] learnopencv. *learnopencv: Stereo Camera Depth Estimation With OpenCV*. URL: https://learnopencv.com/depth-perception-using-stereo-camera-python-c/.

[10] Shree Nayar. *Nayar: First principles of computer vision*. URL: https://fpcv.cs.columbia.edu/.

[11] Plainenglish. *Plainenglish: Stereo block mathcing and depth estimation*. URL: https://python.plainenglish.io/the-depth-ii-block-matching-d599e9372712.

[12] Plainenglish. *Plainenglish: Stereo calibration and rectification*. URL: https://plainenglish.io/blog/the-depth-i-stereo-calibration-and-rectification-24da7b0fb1e0.

[13] Cyrill Stachniss. *Stachniss: Photogrammetry*. URL: https://www.youtube.com/@CyrillStachniss.

[14] Jeremy Steward. *Steward: Modeling camera distortion*. URL: https://www.tangramvision.com/blog/camera-modeling-exploring-distortion-and-distortion-models-part-i.

[15] Paul Mc Whorter. *Mc Whorter: Jetson Nano indtroduction and application*. URL: https://www.youtube.com/@paulmcwhorter.